RV UNIVERSITY

Go, change the world

an initiative of RV EDUCATIONAL INSTITUTIONS

CS2403 COMPUTER NETWORKS

Mini Project Report

Plexo-Enhanced

Seven networking enhancements to the Plexo multi-path download manager

Submitted

By

Harshul Kala (1RUA25CSE0188)

Khushi Krishna (1RUA25CSE0217)

Medha Umesh (1RUA25CSE0257)

Meghna Pant (1RUA25CSE0259)

Under the guidance of:

Prof. Sunil Kumar J

School of Computer Science and Engineering

RV University, Bangalore

RV UNIVERSITY

Go, change the world

an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

CERTIFICATE

Certified that the CS2403 Computer Networks Mini Project work titled Plexo-Enhanced: seven networking enhancements to the Plexo multi-path download manager is carried out by Harshul Kala (1RUA25CSE0188), Khushi Krishna (1RUA25CSE0217), Medha Umesh (1RUA25CSE0257) and Meghna Pant (1RUA25CSE0259), who are bonafide students of the School of Computer Science and Engineering, RV University, Bengaluru, during the year 2025–26. It is certified that all corrections/ suggestions from all the continuous internal evaluations have been incorporated into the project and in this report.

Prof. Sunil Kumar J

Faculty Guide

_____

Program Director

Contents

Plexo-Enhanced at a glance 3
1 Problem statement 4
2 Introduction 4
3 Network diagram 5
4 Configuration setup 7
5 How base Plexo moves bytes 8
6 The seven enhancements 9
6.1 Auto-organize: the folder is decided before the first byte 9
6.2 Learning from network behaviour with Thompson Sampling 11
6.3 A sharp, translucent interface 12
6.4 Network speed test: what each path can really carry 13
6.5 Checksum verification and chunk repair (designed, not implemented) 14
6.6 Link indexing: paste a link, get a searchable catalogue 15
6.7 Pre-download simulation: the answer before the download 17
7 Results 19
7.1 Proof 1: two paths carry one file at the same time 19
7.2 Proof 2: the byte split follows each path's capacity 20
7.3 Proof 3: one pasted link becomes 45 indexed items 20
7.4 Proof 4: real speed test on the real Wi-Fi adapter 21
7.5 Proofs 5 and 6: automated tests 21
8 Limitations and known issues 22
9 Networking concepts in depth 22
9.1 Encapsulation: what Plexo writes and what it only steers 23
9.2 Socket programming, binding and DNS 24
9.3 TCP and HTTP Range are different layers 24
9.4 How a file is divided into chunks 25
9.5 Load balancing, the tail and integrity 26
9.6 Why chunk downloads can need twice the disk, and why Plexo does not 27
9.7 Flow control, limits and fault tolerance 28
9.8 Peer-to-peer and security boundaries 29

9.9 Link indexing, step by step 31

10 Conclusion 31

References 32

Appendix A Where this report meets the rubric 33

Appendix B Test log excerpt 33

Appendix C Short answers for the viva 33

Plexo-Enhanced at a glance

We took Plexo, an open-source download manager that splits one file across several internet connections at once [1], and added seven enhancements. Six are built and tested. One, checksum repair, is designed and not yet coded. Together they let Plexo find files from a pasted link, predict a download before it starts, choose better end-of-download settings from past runs, measure each network, and file the result in the right folder. On our two-path test bench a 300 MB file reached 73.1 MB/s, which is 1.6 times the Wi-Fi path alone, and the bytes split between the paths in the same ratio as their capacity.

7 enhancements: 6 built and tested, 1 designed	**≈7,800** lines of code added (3,510 committed, 4,292 in progress)	**31 / 31** automated checks passing on 5 Nov 2026
73.1 MB/s live combined rate on the bench, 1.6× Wi-Fi alone	**62.5 : 37.5** Wi-Fi : phone byte split, against 64 : 36 capacity	**45** files indexed from one pasted link (ftp.gnu.org)

Measured during this evaluation on the bench in Section 3, or read from the repository. Teal marks the Wi-Fi path and amber the phone path throughout this report.

Credit. The base application, its architecture and every line not listed in Table 3 belong to Anmol Kapil and the Plexo contributors (github.com/anmolkapil/plexo, version 1.0.0-rc.15, licensed open source). Our work is the enhancement code, its tests, and this analysis.

1 Problem statement

A laptop often has two ways to reach the internet, such as campus or home Wi-Fi and a phone tethered over USB, yet an ordinary download uses only one of them, through one TCP connection, and stops making progress whenever that connection slows near the end. Plexo already fixes the core of this at the application layer: it fetches different HTTP byte ranges over sockets pinned to different adapters and joins them on disk [1], [2]. What it still lacks is everything around the transfer. It cannot find files for the user, cannot tell in advance how long a download will take or whether the disk will hold it, keeps the same end-of-download settings for a fast CDN and a throttled mirror, cannot measure what each network can carry, and drops every file into one folder. This project designs, implements and tests seven enhancements that close those gaps without changing Plexo′s transfer guarantees: no byte is ever marked done unless it arrived complete and in place.

2 Introduction

Plexo is an Electron desktop application written in TypeScript and React. Its main process does all networking and disk work; the window only draws state that it receives over typed IPC channels. For an HTTP download it sends a probe request for the first 1,024 bytes, and only a real `206 Partial Content` answer proves the server will split the file. It then plans blocks of 1 to 8 MiB and runs 8 to 32 TCP streams per network, each asking for one block at a time with a `Range: bytes=start-end` header as defined in RFC 9110 [2]. Torrents work the same way at the peer level: every outgoing BitTorrent connection is assigned to the network with the fewest peers [14].

The networking idea that makes this work is source steering. On Linux the routing table sends every packet out of the default route, whatever source address the socket is bound to, so a phone address on a Wi-Fi interface is simply dropped. Plexo calls `setsockopt(SO_BINDTODEVICE)` through libc to bind each socket to the adapter itself (for example `wlp0s20f3` or `usb0`), and races IPv4 against IPv6 connection attempts 250 ms apart, the Happy Eyeballs method of RFC 8305 [3]. Plexo therefore works at layer 7 while steering layers 4 and 3 (Figure 3). It differs from Multipath TCP [4]–[6], which splits one TCP connection across paths inside the kernel and needs support at both ends. Plexo needs nothing from the server except range support, the approach Kaspar et al. measured for HTTP over multiple heterogeneous interfaces [7].

Splitting brings its own problem, the tail. When most blocks are done, the slowest remaining block decides when the file finishes. Plexo answers with hedged requests: an idle stream on another path re-fetches the same block and the first copy to finish wins, the technique Dean and Barroso describe for large services [8]. How early to hedge and how many streams to open are fixed numbers in the base code (2,000 ms and 8 streams). Our learning enhancement turns that fixed choice into a small multi-armed bandit solved with Thompson Sampling [9], [10].

The other enhancements are about the user′s whole job, not just the transfer. A link indexer fetches a page the user pastes, recognises directory listings, RSS feeds, sitemaps, JSON catalogs and Internet

Archive collections, and builds a searchable local index, in the spirit of an incremental web crawler [15]. Because the indexer fetches URLs the user supplies, it rejects loopback and RFC 1918 private addresses [17], a basic defence against server-side request forgery [18]. A pre-download simulation estimates time, speed-up and peak disk use; a speed test measures each path alone and together, following the warm-up practice that measurement studies recommend [12]; auto-organize files each finished download by type; and a design for checksum verification with targeted chunk repair uses SHA-256 and related hashes [19].

This report follows the course template. Sections 3 and 4 give the network diagram and the configuration of every device and interface used. Section 5 explains the base download path we built on, Section 6 covers each enhancement in depth, Section 7 presents the screenshots, measurements and test logs that prove it works, Section 8 lists the limitations we found while testing, including bugs in our own code, and Section 9 walks one download through every networking concept in depth.

3 Network diagram

Figure 1 is the deployment the project targets: one laptop with two independent paths to the internet. The gain only appears when each adapter reaches a different upstream link; Wi-Fi and Ethernet into the same home router share one internet pipe and add nothing [1].

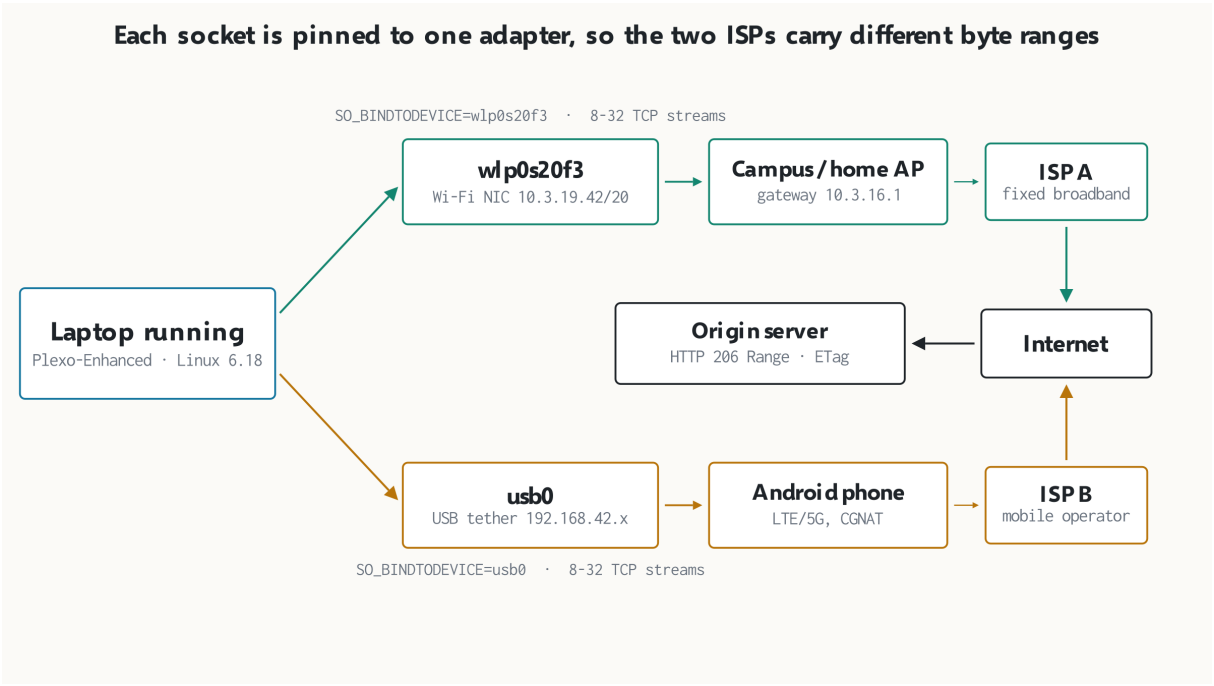


Figure 1. Each socket is pinned to one adapter, so ISP A and ISP B carry different byte ranges of the same file. The origin only has to support HTTP range requests. *Source: authors; adapter names and addresses from the test laptop (ip addr), phone subnet typical of Android USB tethering.*

Figure 2 is the bench used for the results in Section 7. We had one physical adapter on the day of testing, so the second path is emulated with Plexo's own test hook, `PLEXO_E2E_INTERFACES`. Requests on the Wi-Fi path leave from `127.0.0.1`; requests on the phone path leave from the laptop's LAN address `10.3.19.42`. A small Node.js server we wrote, `rangeServer.mjs`, answers range requests for a

300 MiB file and throttles every connection by its source address: 1.6 MB/s for loopback and 0.9 MB/s for LAN. This copies a CDN that limits each connection, which is exactly where multi-stream, multi-path downloading pays off.

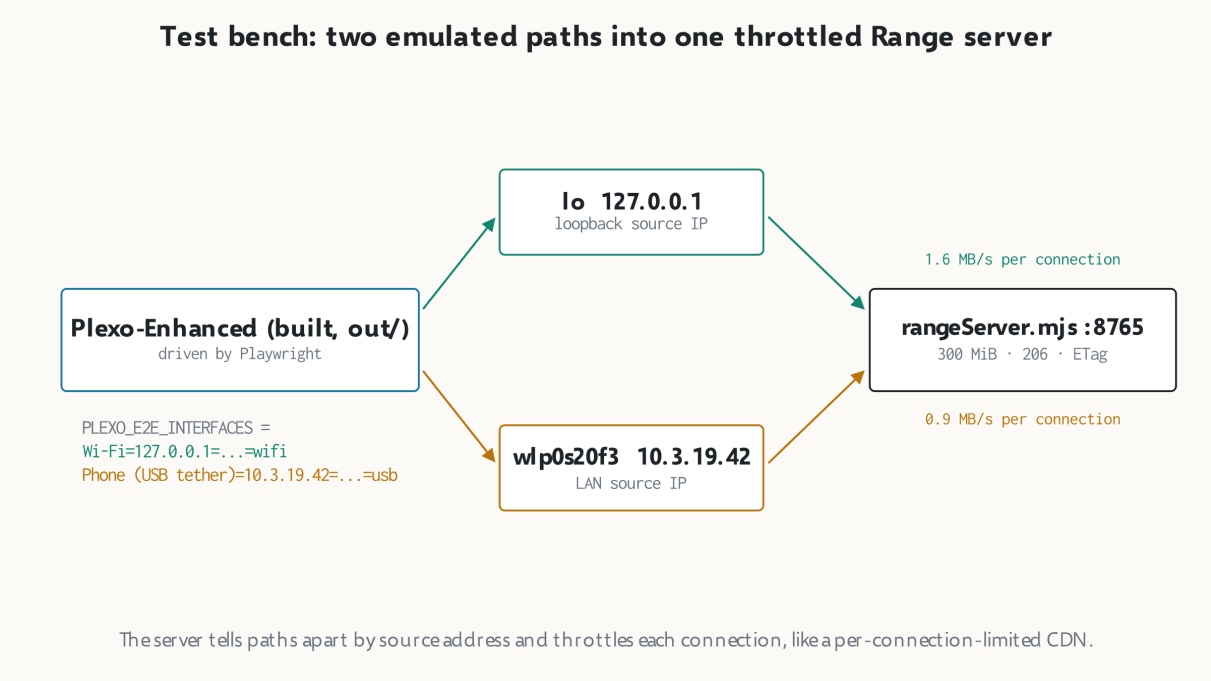


Figure 2. The bench gives two paths with known, different capacities, so the measured split can be checked against a predicted one. *Source: authors; scripts in the project working folder (rangeServer.mjs, drive.cjs).*



Figure 3. Plexo is an application-layer program that controls transport sockets and steers network-layer egress. It reads link-layer adapter names but never builds frames. *Source: authors, from HOW-PLEXO-WORKS.md §2 and src/main/network/.*

4 Configuration setup

Every device and interface used in the evaluation is listed below with its configuration. Addresses are the real ones from the test laptop on 5 November 2026.

Table 1. Devices and interfaces

Device / interface	Role	Configuration
Laptop (host)	Runs Plexo-Enhanced and the test server	NixOS, Linux 6.18.46, x86-64, 32 hardware threads; Wayland session
lo	Wi-Fi path on the bench	127.0.0.1/8; emulated id `Wi-Fi`, kind wifi
wlp0s20f3	Phone path on the bench; real Wi-Fi in the speed test	10.3.19.42/20 by DHCP, gateway on 10.3.16.0/20; emulated id `Phone (USB tether)`, kind usb
Range server	Origin for the bench downloads	rangeServer.mjs on 0.0.0.0:8765; 300 MiB body; ETag "plexo-demo-1"; 206 with Content-Range
Public test file	Origin for the real speed test	https://proof.ovh.net/files/100Mb.dat (206 verified)
Public index source	Origin for link indexing	https://ftp.gnu.org/gnu/hello/ (Apache directory listing)
Phone (deployment)	Second ISP in Figure 1	Android USB tethering, usb0 (RNDIS), mobile data

Table 2. Software and run-time settings

Item	Value	Why
Plexo base	1.0.0-rc.15 (fork of anmolkapil/plexo)	upstream code we extend
Electron	43.1.0 from nixpkgs	the npm Electron 44.4.4 lacks NixOS system libraries
Node.js	24.19.0	build tools, unit tests, range server
Build	`electron-vite build` → out/	same bundle the packaged app uses
UI driver	Playwright _electron	repeatable clicks and screenshots
PLEXO_E2E_BLOCK_BYTES	4,194,304 (4 MiB cap)	plan gives 128 blocks of 2.3 MB for 300 MiB
PLEXO_E2E_SPEEDTEST_STAGE_MS	4,000	shorter stages for the bench (default 10,000)
PLEXO_E2E_SPEEDTEST_WARMUP_MS	800	warm-up excluded from the rate (default 2,000)
Streams per network	Auto (start 8, grow to 32)	default concurrency controller
Auto organize	on, default buckets	Archives, Videos, Documents, Programs, Others
Learning, act on tail	off (default)	learning records only, as shipped

Two set-up details cost us time and are worth recording for anyone repeating the work. First, the shell's `LD_LIBRARY_PATH` pointed at a newer ALSA library than Electron's C library, so every Electron launch failed with `GLIBC_2.43 not found` until the variable was unset. Second, the emulated adapter names are not real kernel devices. A speed test to a public server on those names fails with `errno 19`

(ENODEV) because `SO_BINDTODEVICE` refuses an unknown device. That failure is the socket pinning working as designed, and it is why the real speed test in Section 7 runs on the real adapter `wlp0s20f3`.

5 How base Plexo moves bytes

Every enhancement hooks into the pipeline in Figure 4, so its rules matter. The probe asks for bytes 0 to 1,023 rather than sending `HEAD`, because many servers advertise `Accept-Ranges` on HEAD and then ignore the range on GET. The planner aims for 64 blocks per network (32 streams × 2 blocks each) and clamps each block between 1 MiB and 8 MiB. The upper cap limits tail cost, since a hedge re-fetches a whole block; the lower cap keeps request overhead small.

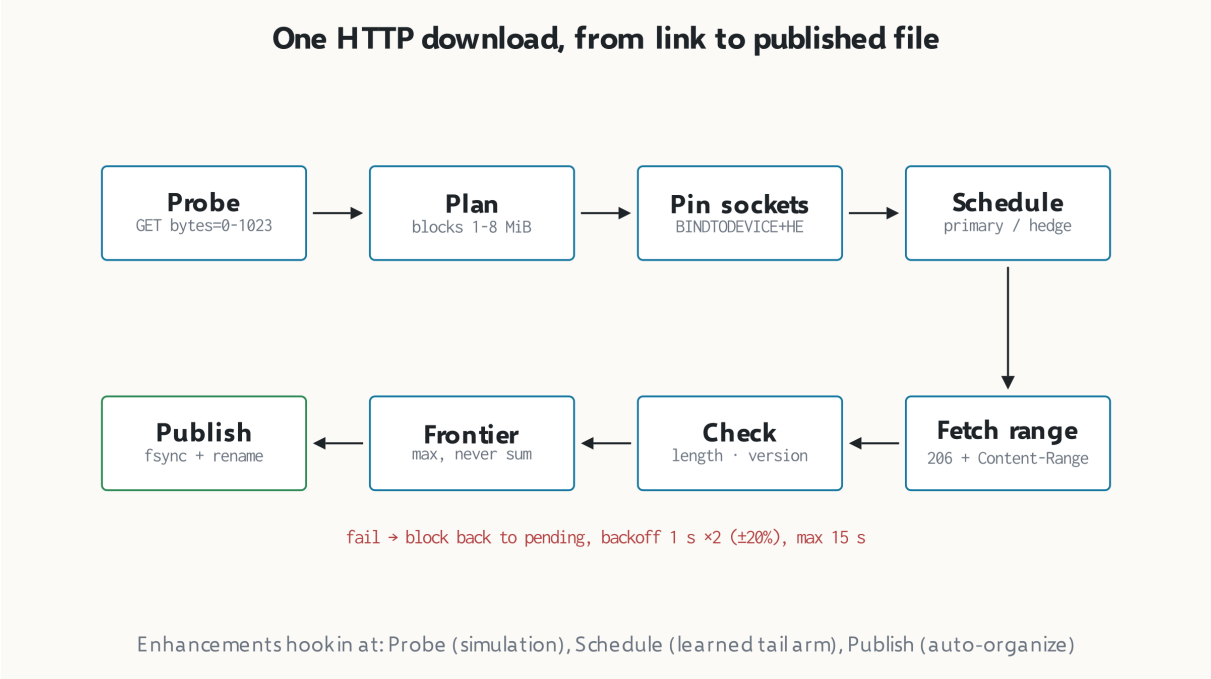


Figure 4. A block becomes done only after its length, its Content-Range and its file version all check out. The enhancements attach at three points shown along the bottom. *Source: authors, from src/main/download/probe.ts, plan.ts, scheduler.ts, chunkDownloader.ts, downloadFile.ts.*

Each response is checked before a single byte is written. A `200` is accepted only for a block that starts at byte 0, because otherwise the server ignored the range and would write the start of the file into the middle. The `ETag`, `Last-Modified` and total size are compared with the version accepted at the start; a changed size proves the file was replaced. Bytes go to their exact offset in a staging file `name.plexo` through a separate file descriptor per attempt, and progress is tracked as a frontier, the highest secured offset in each block, never a sum, so two racing copies are not counted twice. When every block is done the file is flushed with `fsync` and renamed into place in one step.

Speed limits use token buckets. When a bucket is empty Plexo stops reading the socket; the kernel receive buffer fills, the advertised TCP window shrinks, and the sender slows down by itself. No packet is dropped. This is TCP flow control doing the work, the same feedback loop Jacobson described for congestion avoidance [13].

6 The seven enhancements

Table 3 lists the seven enhancements, where the code lives and how each was verified. The subsections that follow explain each one: the problem it solves, how it works, and what it changes on the network.

Table 3. The seven enhancements and their status

#	Enhancement	Main code	Status	Verified by
1	Auto-organize downloads	download/organize.ts, scripts/organize.py, OrganizeDialog.tsx	built	5 unit tests, screenshot
2	Learning from network behaviour	download/learning/ (bandit, profiles, context, store)	built	6 unit tests, screenshot
3	Sharp, translucent interface	windowMaterial.ts, main.css, MaterialToggle.tsx	built	1 unit test, screenshots
4	Network speed test	download/speedTest.ts, SpeedTestDialog.tsx	built	1 unit test, real run
5	Checksum verification and chunk repair	feature-05 design document	**designed**	design review only
6	Link indexing and content discovery	main/discovery/ (8 files), Sources/Discover screens	built	13 checks, live index
7	Pre-download simulation	utils/simulationEngine.ts, PreDownloadSimulationDialog.tsx	built	5 checks, screenshot

6.1 Auto-organize: the folder is decided before the first byte

Base Plexo saved every file into one destination folder. Enhancement 1 picks a category folder from the file extension at the moment the download is created, so the staging file `.plexo` is already in the final folder and publishing stays a same-folder rename, with no copy and no second file's worth of disk space. `extensionOf()` takes the text after the last dot in lower case (so `archive.tar.gz` gives `gz`). `categoryForFileName()` walks the buckets in order and returns the first whose list contains it; the `Others` bucket has an empty list and catches the rest.

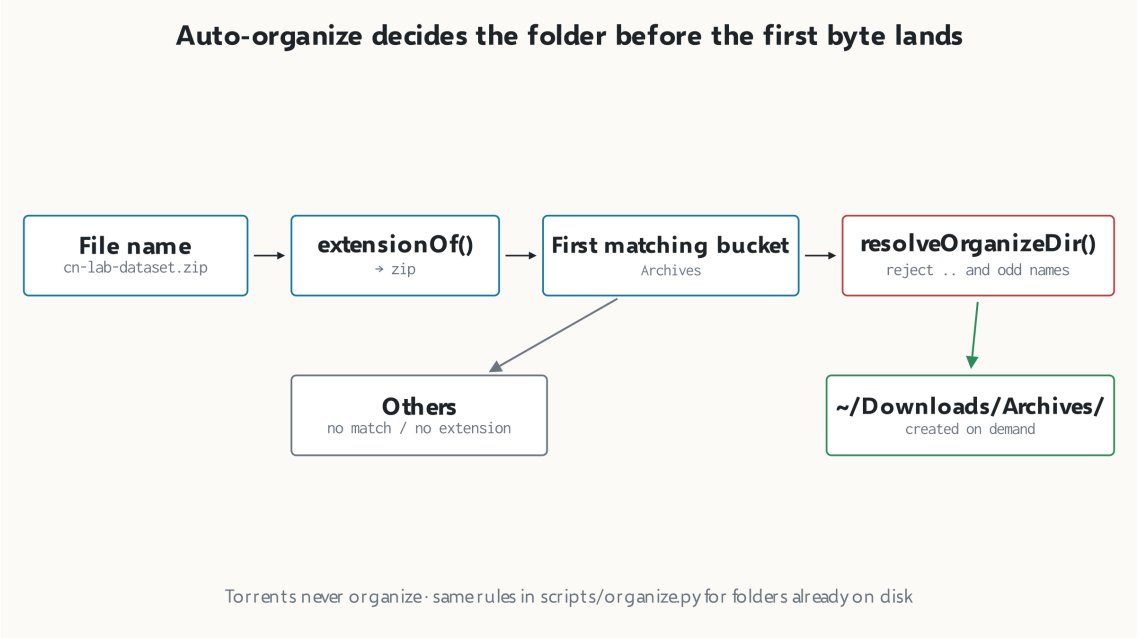


Figure 5. Organizing is a pure decision on the name, made once, before any network traffic. Path checks keep a user-edited folder from escaping the download directory. *Source: authors, from src/main/download/organize.ts.*

Users can rename a bucket, change its extension list or point it at another folder. Because that folder string comes from the user, `resolveOrganizeDir()` rejects any absolute path that contains `..` and any relative name outside `[A-Za-z0-9 _.-]{1,64}`, falling back to the default subfolder. Torrents are never organized, since a torrent defines its own folder layout. The same rules ship as `scripts/organize.py` for folders that already hold loose files; the app passes its current buckets to the script so both always agree. Figure 6 shows the preview line in the New download dialog, which tells the user where the file will land before they press Download.

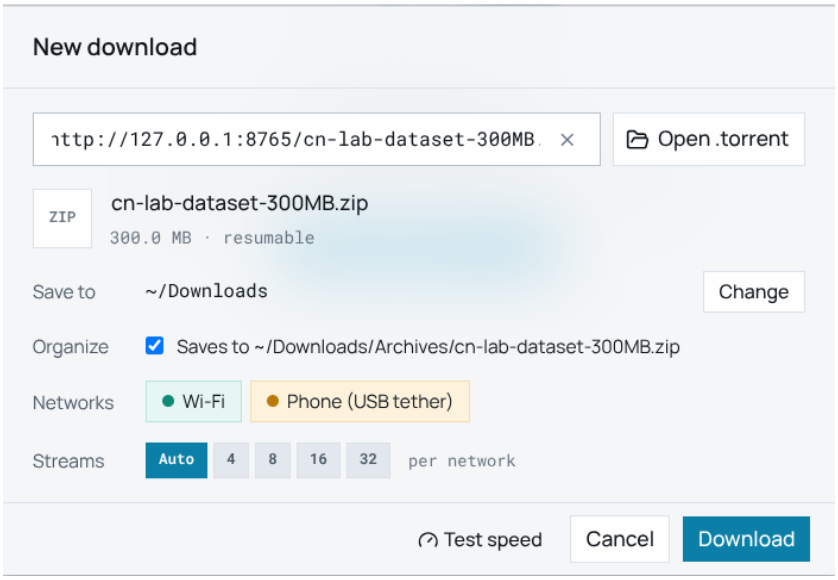


Figure 6. The New download dialog previews the organized path: a .zip goes to ~/Downloads/Archives/. The probe has already confirmed 300.0 MB and range support (resumable). *Source: screenshot of Plexo-Enhanced, bench, 5 Nov 2026.*

6.2 Learning from network behaviour with Thompson Sampling

Some servers limit each connection. Near the end of a download, one slow block on such a server can hold the whole file while idle streams wait, because the base rules (start 8 streams, hedge after 2 s) assume an average server. Enhancement 2 keeps a local memory of how each host and each network behaved and uses it to choose one of three tail strategies, called arms: conservative (4 streams per network, hedge after 4,000 ms), standard (8 streams, 2,000 ms, the base default) and aggressive (16 streams, 1,000 ms).

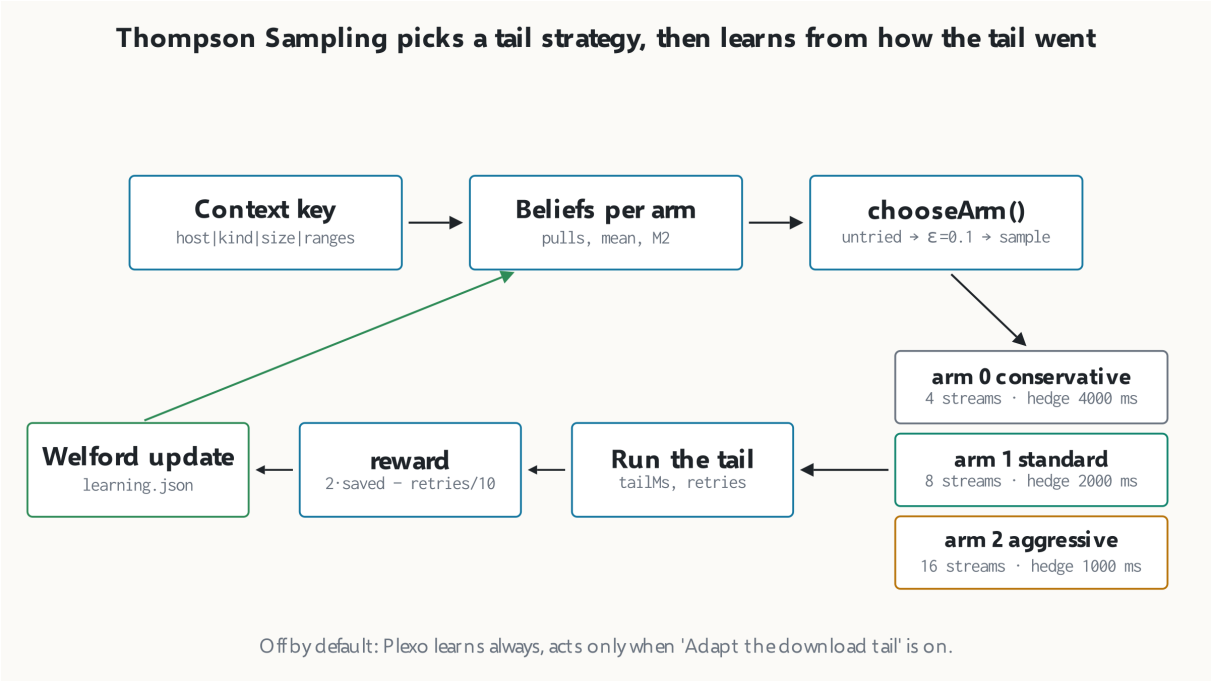


Figure 7. After each finished download the tail is scored, the chosen arm's belief is updated, and the next download in the same context samples from the updated beliefs. *Source: authors, from src/main/download/learning/bandit.ts, profiles.ts, context.ts.*

The learning context is the key `host|network kind|size band|range support`, for example `ftp.gnu.org|wifi|s|r`. It is narrow enough to matter (a host that throttles phones but not Wi-Fi learns two lessons) and broad enough to repeat, which learning needs. For each context the store keeps, per arm, the number of pulls, the mean reward and the running sum of squared deviations, updated with Welford's one-pass method [11] so no history list is stored. The reward of a run is `clip(2 × (expectedTail − tail) / expectedTail − min(retries/10, 1), −2, +2)`, and a failed run scores −2. A fast tail scores near +2; a tail that needed many retries scores below zero.

`chooseArm()` first tries every untried arm once, picked uniformly. After that, with probability ε = 0.1 it explores at random; otherwise it draws one sample per arm from a normal distribution with that arm's mean and standard deviation (variance floored at 0.01) and plays the largest sample. That is Gaussian Thompson Sampling: an arm wins as often as it is likely to be best, so a clearly better arm is chosen most of the time while an uncertain one still gets tried. Agrawal and Goyal proved that this strategy's regret grows only logarithmically [9], and Chapelle and Li showed it matches or

beats UCB-style methods in practice [10]. Host profiles also keep exponential moving averages ($\alpha = 0.3$) of throughput, time to first byte and tail length.

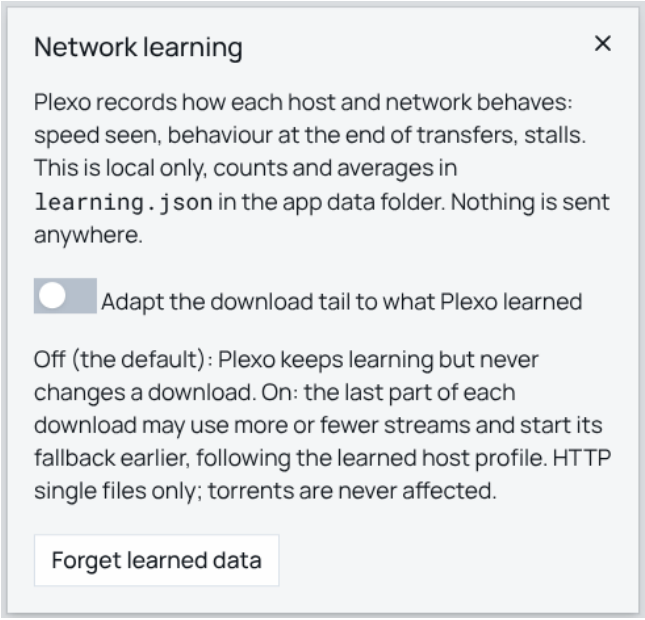


Figure 8. Learning is local and opt-in. Plexo records profiles in learning.json but only changes a download's tail when the user switches on Adapt the download tail. *Source: screenshot of Plexo-Enhanced, 5 Nov 2026.*

The safety rule is that the learner only advises. The deterministic engine still runs every download inside its hard caps (32 streams per network, at most 2 hedges per block), the feature is off by default, HTTP single files are the only downloads it can touch, and a Forget button wipes the store. The store itself is bounded with least-recently-used eviction and recovers from a corrupted file by starting fresh, both covered by unit tests.

6.3 A sharp, translucent interface

Enhancement 3 changes how the window looks without changing what it does. Every rounded corner became a 0 px corner, so structure comes from hairlines and spacing, and the window gained three material levels: opaque, glass panels and desktop glass. Most of this is CSS keyed on a `data-material` attribute. Seeing the real desktop through the window is a platform option: Windows 11 uses `setBackgroundMaterial('acrylic')`, macOS uses `vibrancy`, and Linux needs a window created with `transparent: true`. That is a creation-time option, so on Linux switching to or from desktop glass rebuilds the window. Downloads live in the main process, so a rebuild loses nothing; we saw this during testing when the Playwright session lost its page handle on the toggle, exactly as the code intends.

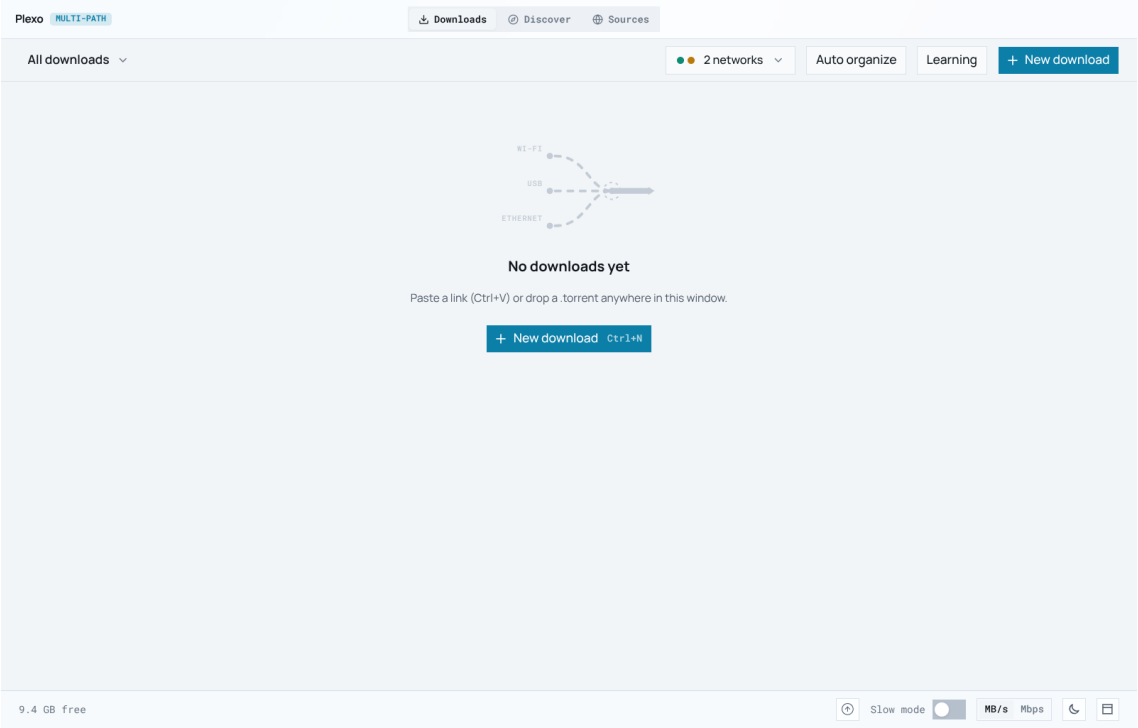


Figure 9. The refreshed main window: sharp edges, interface-coloured network dots (teal Wi-Fi, amber phone) and the empty state that tells the user what to do next. *Source: screenshot of Plexo-Enhanced, bench, 5 Nov 2026.*

6.4 Network speed test: what each path can really carry

Before this enhancement the only way to learn a network's speed was to start a real download. Enhancement 4 measures every selected network before any download exists, using the same machinery a download uses: real HTTP GET streams, each pinned to one adapter through `StreamConnection` and `SO_BINDTODEVICE`, reading the body and discarding it. Nothing touches the disk, so the test costs data and time, never space.

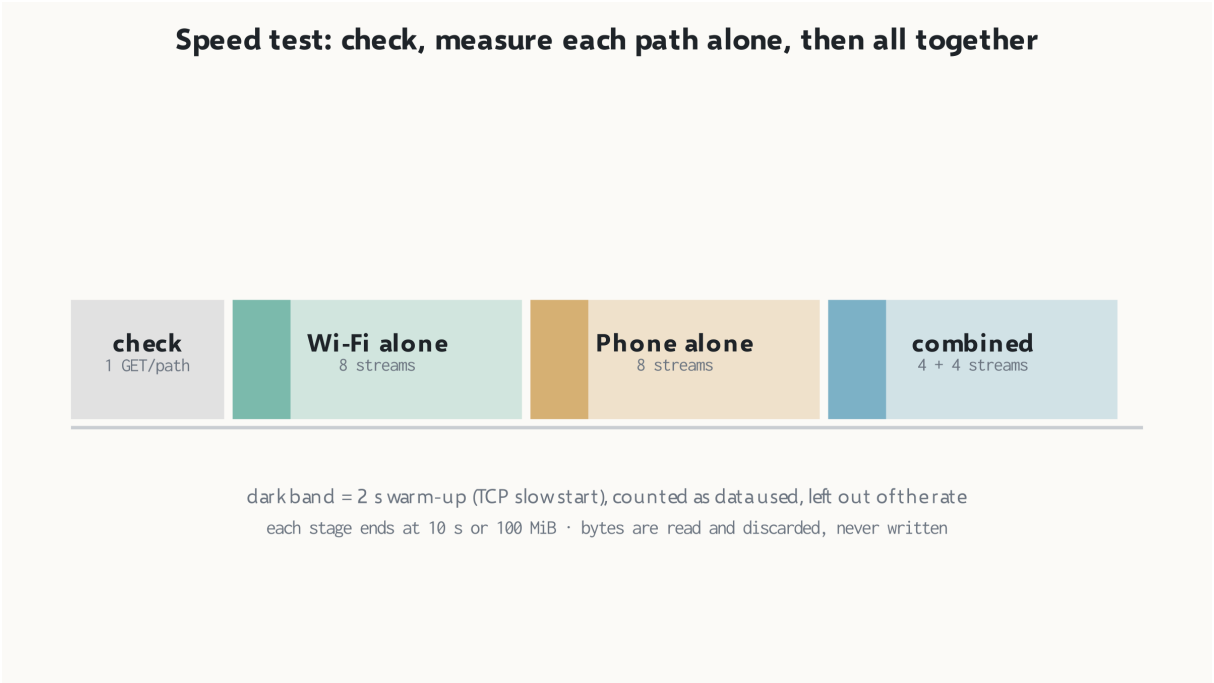


Figure 10. Every stage uses the same total of 8 streams, so the numbers compare: a path alone gets all 8, and the combined stage shares them out. *Source: authors, from src/main/download/speedTest.ts and shared/types.ts constants.*

The run has three stages. First, one request per network checks reachability, so a network that cannot reach the URL is named before any time is spent on it; errors are translated into words a user can act on, such as 'No route from this network' for `ENETUNREACH`. Second, each reachable network is measured alone. Third, all reachable networks are measured together. Every stage begins with a 2 s warm-up whose bytes count as data used but not toward the rate, because the first second of a TCP connection is slow start, not steady capacity [12], [13]. A stage ends after 10 s or 100 MiB, whichever comes first. The dialog states the method's limit plainly: networks behind one upstream link share it, so together they can add up to less than the sum, and any single run describes one moment.

6.5 Checksum verification and chunk repair (designed, not implemented)

A file built from chunks over several networks has a new failure mode: one link can return a short or damaged body with a `200 OK` while the others are fine. Base Plexo already rejects wrong lengths and shifted ranges, but it cannot detect wrong bytes of the right length. Enhancement 5 is our design for that case. After the last block, the assembled file is hashed with Node's built-in `crypto` (MD5, SHA-1 or SHA-256 [19]) and compared with a checksum the user supplies or the page publishes. On a mismatch, per-block hashes locate the bad ranges, and only those are re-fetched, preferring the path the learning bandit ranks highest, with at most three repair rounds before the user is asked to act.

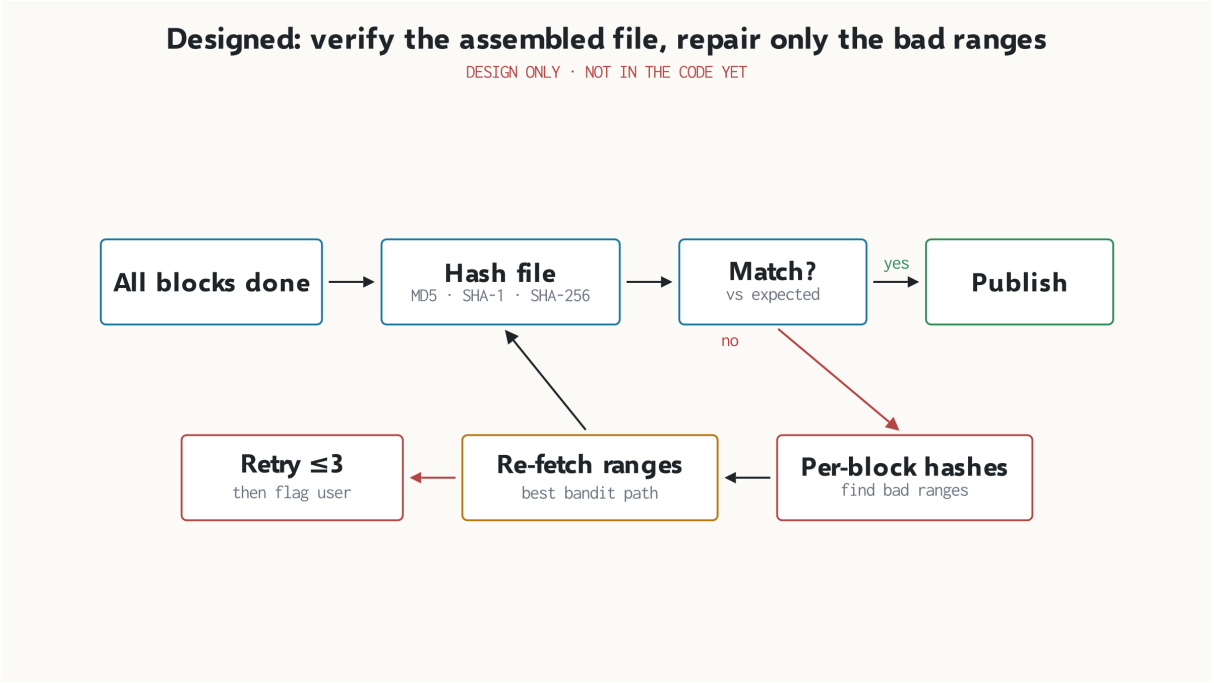


Figure 11. The repair loop re-downloads only damaged ranges, not the whole file. This flow exists as a design document; no code for it is in the repository yet. *Source: authors, from plexo-feature-docs/feature-05-checksum-verification.html.*

The data model already has room for it: indexed items carry an optional `hash: { algorithm, value }` field in `shared/sourceTypes.ts`. One honest weakness of the design is that a whole-file hash cannot point to a bad block by itself. Locating the damage needs per-block reference hashes, which a plain

HTTP server does not publish. A torrent has them in its piece hashes. For HTTP, the repair step would have to re-fetch blocks and compare them, or rely on a manifest the publisher provides.

6.6 Link indexing: paste a link, get a searchable catalogue

This is the enhancement the brief asked for by name. The user pastes a URL on the Sources screen; Plexo fetches it, works out what kind of page it is, extracts every downloadable item, and keeps them in a local index the Discover screen can search. Figure 12 shows the full path from the pasted link to a bonded download.

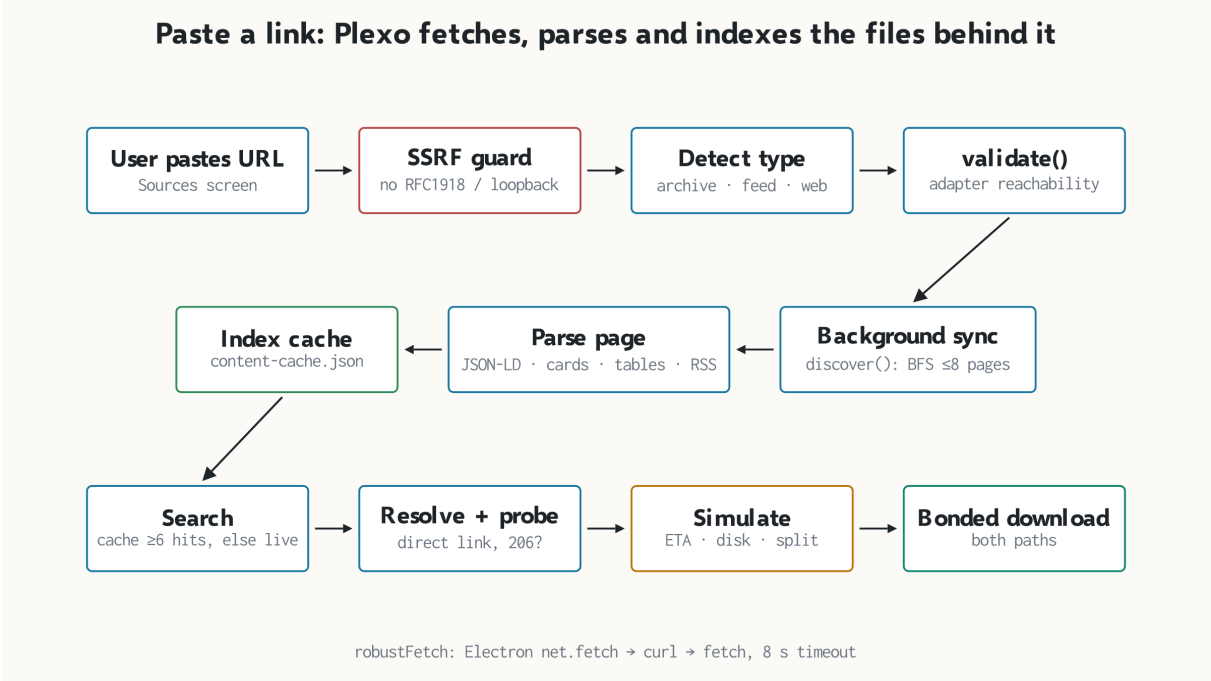


Figure 12. Indexing runs in the main process, never in the window. The SSRF guard runs before any request leaves the machine. *Source: authors, from src/main/discovery/sourceIndex.ts, fetcher.ts, htmlParser.ts, adapters/.*

`SourceIndexManager.addSource()` first calls `isPrivateOrLocalUrl()`. It refuses any scheme other than http and https, the names `localhost`, `.local` and `.internal`, and the address blocks 10/8, 127/8, 169.254/16, 172.16/12 and 192.168/16 [17]. Without this check, a pasted link could make the app probe the user's router admin page or a cloud metadata endpoint, the classic server-side request forgery pattern [18]. Next, `detectSourceType()` picks an adapter: any archive.org address uses the Internet Archive adapter (its advanced-search JSON API), a path ending in `.json` uses the catalog-feed adapter, and everything else uses the universal web adapter. The adapter's `validate()` confirms the source answers, the source is saved to `content-sources.json`, and a background sync starts at once.

The universal adapter is a small breadth-first crawler. It starts from the pasted page, follows pagination (`rel="next"`, numbered links, 'Next' and '»') and catalogue sub-pages, and stops at 8 pages or 350 items, the politeness and scope limits any incremental crawler needs [15]. On each page `htmlParser.ts` tries its strategies in order: JSON-LD structured data, Next.js `__NEXT_DATA__`, card containers with data attributes, `<article>` cards, WordPress entries, table listings such as Apache

directory indexes, plain link lists with sizes, and finally direct file links. RSS 2.0 items and XML sitemaps are parsed too. If a page yields nothing, it probes the usual feed endpoints: `/wp-json/wp/v2/posts`, `/feed/`, `/rss` and `/sitemap.xml`.

Every fetch goes through `robustFetch()`, which tries Chromium's network stack via Electron's `net.fetch` first (HTTP/2, the system's TLS), then a `curl` subprocess, then Node's `fetch`, each with an 8 s timeout. Search is cache-first: if the local index already has at least six title matches it answers at once, otherwise it asks the adapter live and merges the results without duplicate page URLs. When the user picks an item, `resolveContentDownload()` finds the direct file link and runs Plexo's normal probe on it, so an indexed item enters the same checked pipeline as a pasted download link.

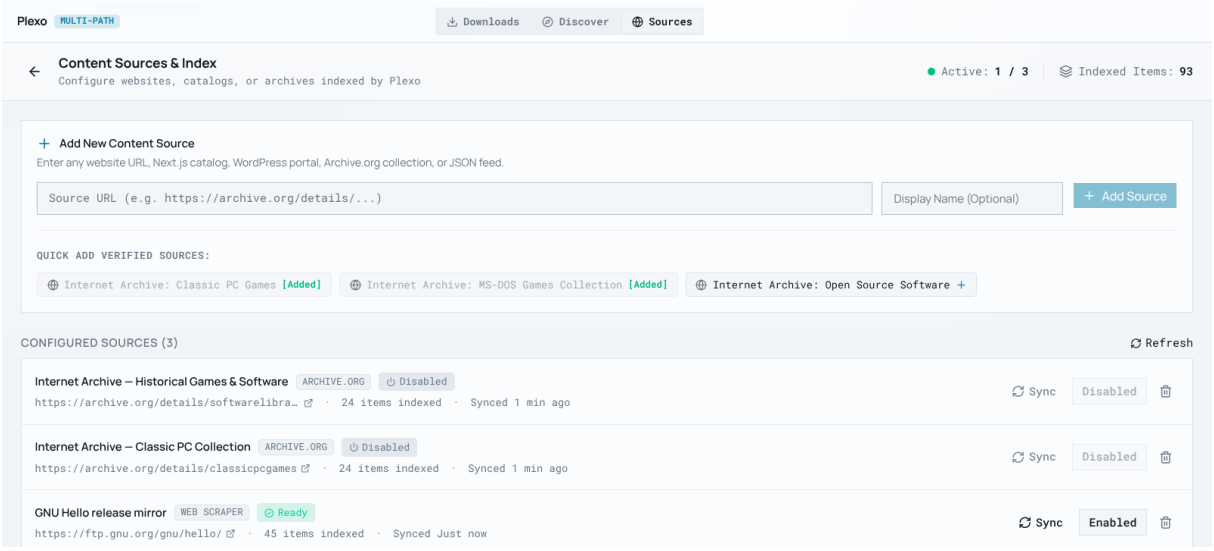


Figure 13. One pasted link, https://ftp.gnu.org/gnu/hello/, became a ready source with 45 indexed items, synced seconds earlier. The two Internet Archive defaults were switched off for this run. *Source: screenshot of Plexo-Enhanced, live internet, 5 Nov 2026.*

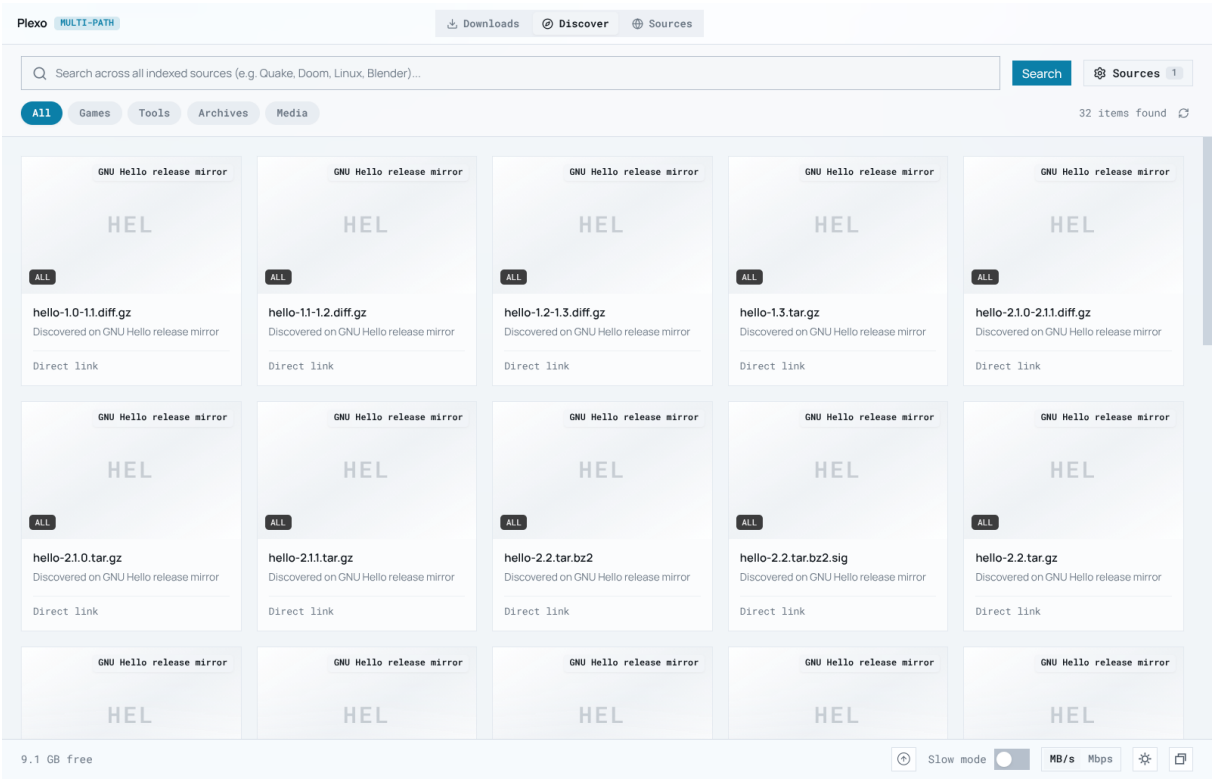


Figure 14. The Discover screen searches the index. Every GNU Hello release and patch file from the directory listing appears as a card with its source and a direct-link badge. *Source: screenshot of Plexo-Enhanced, live internet, 5 Nov 2026.*

6.7 Pre-download simulation: the answer before the download

When the user opens an indexed item and presses Simulate & Download, Enhancement 7 shows what will happen before any byte moves: combined speed, speed-up over the fastest single path, estimated time, the share of chunks per network, and whether the disk can hold the download while it is being assembled. The model is deliberately simple arithmetic (Figure 15), and the report states that plainly: the per-path speeds are assumed defaults for the adapter kind, capped by any speed limit the user set, not measurements.

The pre-download simulation is arithmetic on assumed speeds, not a measurement

```
speed_i      = min(user limit, default for kind: wifi 28, eth 60, other 20 MB/s)
bonded       = 0.92 × Σ speed_i    (0.92 = assumed efficiency)
speedup      = bonded ÷ fastest single path
share_i      = speed_i ÷ Σ speed_i    (split shown as a bar)
ETA          = file size ÷ bonded
peak disk    = 1.08 × size → safe | low margin (<5 GiB) | insufficient
```

Figure 15. The simulation multiplies assumed speeds by a 0.92 bonding efficiency and sizes peak disk use at 1.08 times the file. It predicts; the speed test measures. *Source: authors, from src/renderer/src/utils/simulationEngine.ts.*

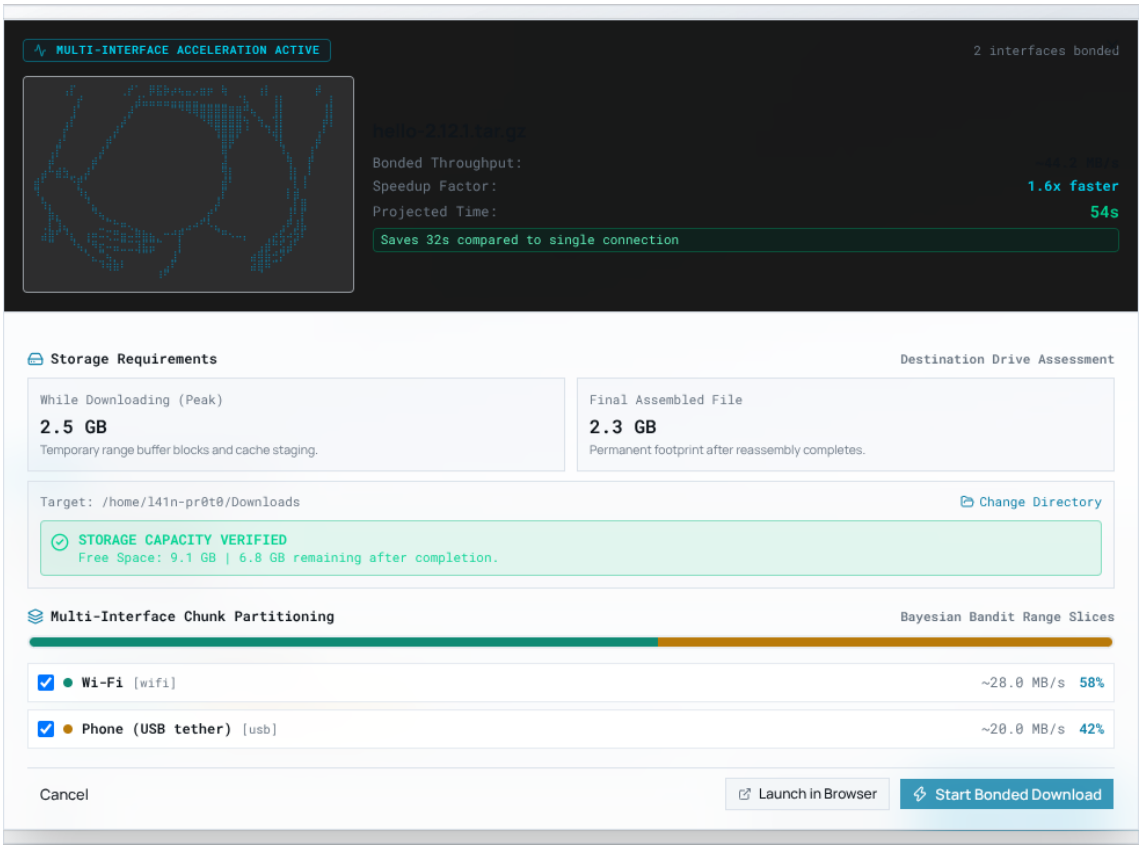


Figure 16. Simulation for an indexed file on two paths: 1.6× faster, 54 s projected, 32 s saved; peak 2.5 GB staging against 9.1 GB free, so storage is verified. Dialog widened with an injected style for legibility; see Section 8. *Source: screenshot of Plexo-Enhanced, bench, 5 Nov 2026.*

The storage check is the part we consider most useful. It flags a download as insufficient when free space is below the peak estimate, and as low margin when less than 5 GiB would remain. The check runs before the download is created, so the user is warned before a long transfer fails near the end

with a full disk. Base Plexo already refuses to start when space is short; the simulation shows the margin in advance.

7 Results

We give six proofs: two from a bonded download, one from link indexing, one from a real speed test, and two from automated tests. All were captured on 5 November 2026 from the built application, driven by Playwright so they can be repeated.

7.1 Proof 1: two paths carry one file at the same time

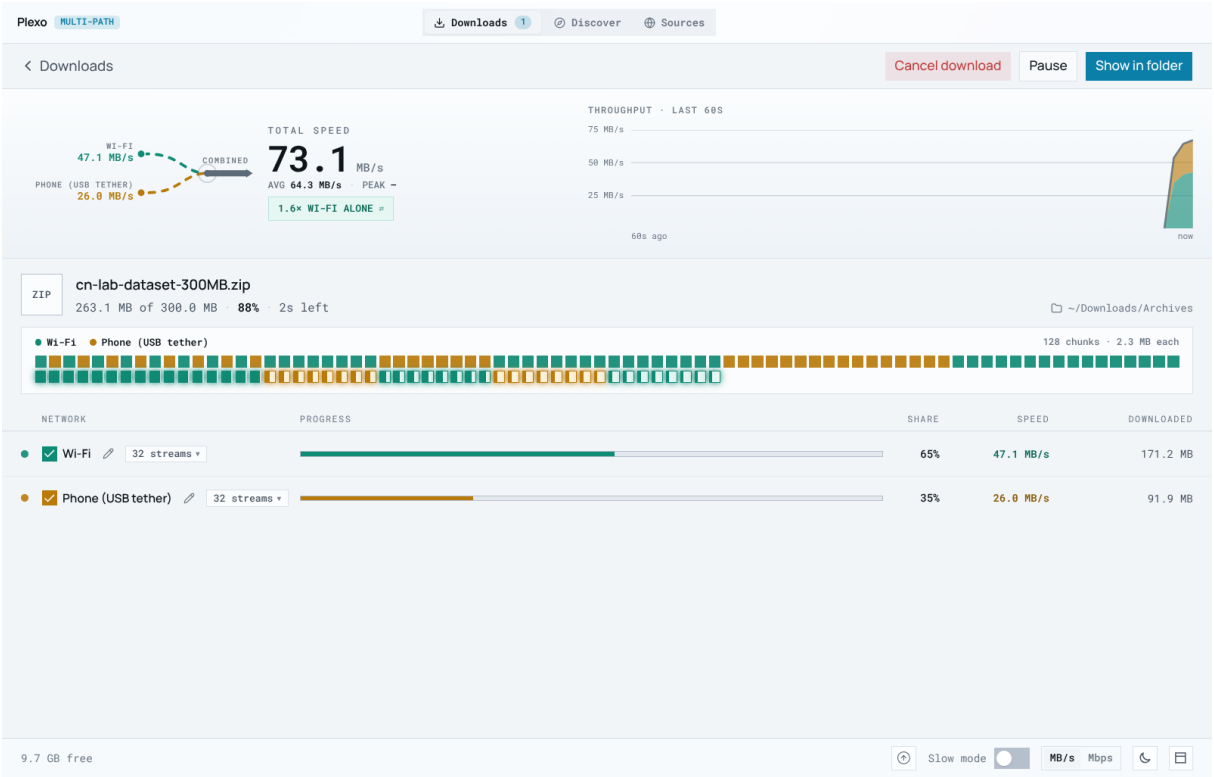


Figure 17. Mid-download on the bench: Wi-Fi 47.1 MB/s and phone 26.0 MB/s give 73.1 MB/s combined, labelled 1.6× Wi-Fi alone. The block grid shows each 2.3 MB chunk coloured by the path that fetched it; 32 streams run on each path. *Source: screenshot of Plexo-Enhanced, bench, 5 Nov 2026.*

The numbers can be checked against the bench. Thirty-two streams at 1.6 MB/s allow at most 51.2 MB/s on the loopback path, and 32 at 0.9 MB/s allow 28.8 MB/s on the LAN path. The observed 47.1 and 26.0 MB/s are 92% and 90% of those ceilings, with the remainder lost to timer granularity in the test server and to streams still ramping up.

7.2 Proof 2: the byte split follows each path's capacity

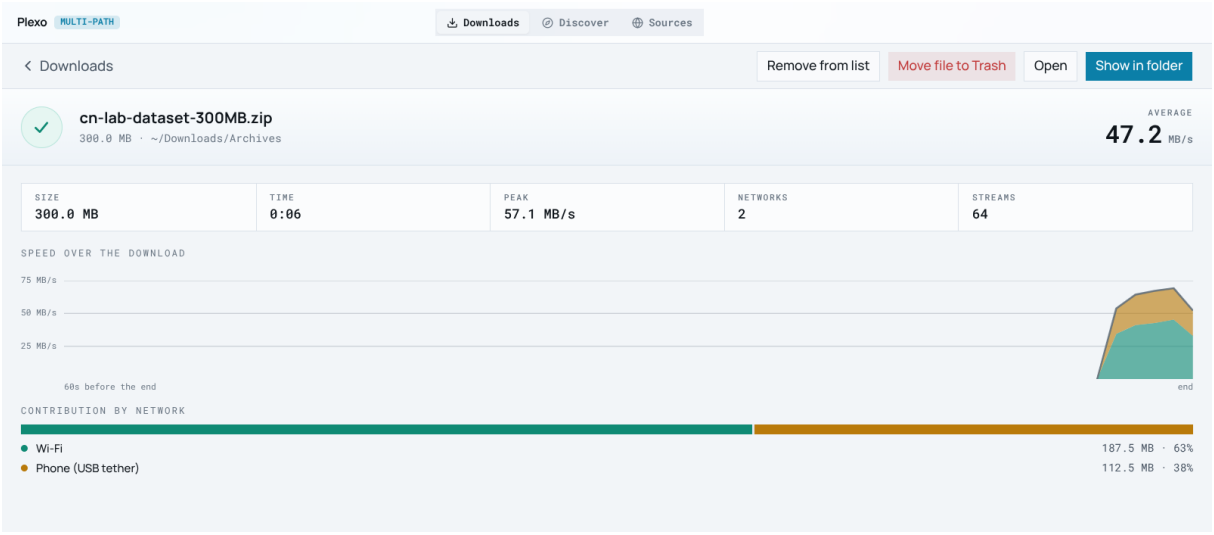


Figure 18. The completed 300.0 MB file: 6 s, average 47.2 MB/s, peak 57.1 MB/s, 64 streams over 2 networks, 0 retries, filed in ~/Downloads/Archives by auto-organize. *Source: screenshot of Plexo-Enhanced, bench, 5 Nov 2026.*

Table 4. Bench download results, 300 MiB file, two emulated paths

Measure	Wi-Fi path	Phone path	Both
Per-connection limit (server)	1.6 MB/s	0.9 MB/s	–
Ceiling at 32 streams	51.2 MB/s	28.8 MB/s	80.0 MB/s
Live rate, mid-download	47.1 MB/s	26.0 MB/s	73.1 MB/s
Bytes delivered	187.5 MB	112.5 MB	300.0 MB
Share delivered	62.5%	37.5%	100%
Share predicted from capacity	64.0%	36.0%	100%
Average over whole run (incl. ramp-up)	–	–	47.2 MB/s
Retries	–	–	0

Source: Plexo-Enhanced completion screen and live screen; capacity from rangeServer.mjs settings.

The split is the result we care about most. Plexo has no knowledge of the server's limits, yet the phone path delivered 37.5% of the bytes when its capacity share was 36%. The scheduler hands a new block to whichever stream frees up first, so faster paths simply ask more often. The UI rounds both shares up and shows 63% and 38%; the exact values come from the byte counts.

7.3 Proof 3: one pasted link becomes 45 indexed items

Figures 13 and 14 are the proof for link indexing. The GNU directory page was fetched, recognised as a table listing, parsed into 45 items (releases, patches and signatures), saved to the cache and searched. Figure 16 shows the same item flowing into the simulation, ready to become a bonded download. The SSRF guard was proved by the unit test: adding `http://127.0.0.1`, `http://192.168.1.1` and similar addresses raised 'Private, loopback, or non-HTTP URLs are prohibited'.

7.4 Proof 4: real speed test on the real Wi-Fi adapter

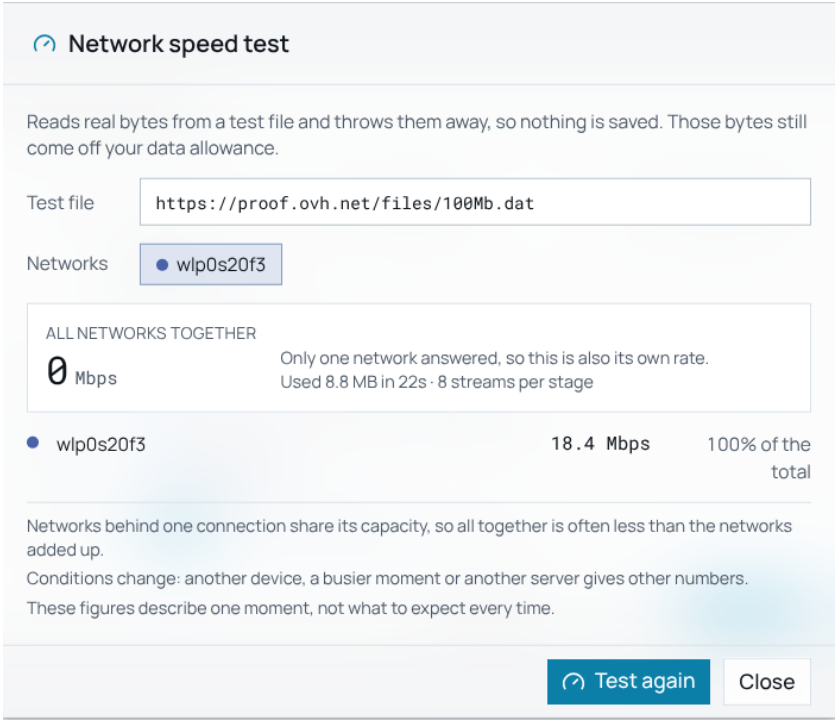


Figure 19. A real speed test on wlp0s20f3 to proof.ovh.net measured 18.4 Mbps and used 8.8 MB in 22 s. The combined line reads 0 Mbps, a bug we report in Section 8. *Source: screenshot of Plexo-Enhanced, campus Wi-Fi, 5 Nov 2026.*

The first public file we tried, Cloudflare's speed endpoint, answered 403 to this client, and the dialog said so in plain words ('The server answered 403') rather than showing an empty rate. We switched to the OVH test file after checking it with `curl -r 0-1023`, which returned 206.

7.5 Proofs 5 and 6: automated tests

Table 5. Automated checks run with npm test on 5 Nov 2026

Suite	What it checks	Result
verify-all-features (node:test)	organize edge cases, path traversal, Python dry run; bandit exploration, reward, Welford; context keys; store LRU and corruption; window material; speed-test cancel	13 / 13 pass
sourceIndex	JSON-LD, __NEXT_DATA__, cards, Livewire cards, mirror links, tables, pagination, RSS and sitemap; SSRF rejection; enable/disable; persistence; removal; adapters	13 / 13 pass
verify-simulation	duration formatting, bonding speed-up and peak storage, storage safety states, single-path baseline, ASCII art asset	5 / 5 pass
Total		31 / 31 pass

Source: console output of npm test, Appendix B.

The project also carries Playwright end-to-end specs for organize, learning, speed test and the interface (`e2e/*.spec.ts`). We did not run that suite in this evaluation: its fixture launches the npm Electron build, which cannot load on NixOS without the system libraries noted in Section 4. The screenshots in this section come from the same app driven by Playwright against the Nix Electron instead.

8 Limitations and known issues

Testing the app end to end turned up the issues below. We list them because a fair evaluation needs them, and because each one is a concrete next task.

Table 6. Known issues found during this evaluation

#	Area	Issue	Effect / fix
1	Checksum (5)	Designed only; no code.	Implement with node:crypto; needs per-block reference hashes.
2	Speed test (4)	Combined row shows 0 Mbps when one network answers.	Show the single network's rate.
3	Simulation (7)	Uses assumed speeds per kind, not speed-test or learning data; unknown sizes default to 2,500,000,000 bytes.	Feed measured rates and the probe's real size.
4	Simulation (7)	Label says 'Bayesian Bandit Range Slices' but the split is proportional to assumed speed.	Rename the label, or wire in the bandit.
5	Indexing (6)	Universal adapter tags every item as 'game'.	Classify by extension, as auto-organize does.
6	Indexing (6)	No robots.txt check [16]; browser-like User-Agent and Sec-Fetch headers; rateLimitMs field unused.	Honest UA, honour RFC 9309, per-host delay.
7	Indexing (6)	SSRF guard checks the host text only: no IPv6 ULA/link-local, no DNS rebinding check, redirects followed by curl not re-checked.	Resolve first, check every IP and every hop.
8	Indexing (6)	Index fetches use the default route, not pinned adapters.	Acceptable; pages are small.
9	Interface (3, 6, 7)	Details and simulation dialogs clip on the right; the organize dialog is taller than the window.	Responsive max-width and inner scroll.
10	Bench	Both emulated paths end on one machine and one Wi-Fi link.	Repeat with a real phone tether and two ISPs.

9 Networking concepts in depth

This section follows one download through every networking mechanism Plexo uses, in the order the bytes meet them. Each subsection has a matching short video without audio in the folder Plexo-Enhanced_Clips, and the figure is that video′s final frame. Table 7 maps the vocabulary first.

Table 7. Computer-networks concepts and where Plexo uses them

Concept	Meaning in plain words	Where it appears in Plexo
Multi-homing	one computer on several independent networks	Wi-Fi, USB phone tether and Ethernet at once
TCP	reliable, ordered byte stream (layer 4) [20]	every HTTP stream and every BitTorrent peer
HTTP Range	ask for bytes s to e of a resource [2]	every block is one Range GET answered by 206

Socket programming	creating and controlling connections in code	Node net and tls sockets; libc calls through koffi
Source address binding	choose the local IP of a connection	localAddress, for example 10.3.19.42
Interface binding	restrict a socket to one network device	setsockopt SO_BINDTODEVICE on Linux
DNS resolution	host name to IPv4 and IPv6 addresses	lookup(all) with a deadline, Happy Eyeballs [3]
Load balancing	spread work across paths	a free stream pulls the next pending block
Flow control	the receiver slows the sender	token bucket, pause, TCP window falls to zero
Connection pooling	reuse a warm TCP and TLS connection	keep-alive agent per stream
Fault tolerance	keep working when a path fails	blocks re-queue and resume from the frontier
Peer-to-peer	download from many peers at once	BitTorrent peers spread across networks [14]
Network security	TLS [21], CORS, localhost limits, SSRF	bridge on 127.0.0.1; SSRF guard on sources
Not used	WebSockets, proxies	downloads are direct, pinned HTTP/TCP and BitTorrent/TCP

9.1 Encapsulation: what Plexo writes and what it only steers

A Range request for block 1 leaves the laptop wrapped five times. Plexo writes the innermost part, the HTTP request line and its `Range: bytes=2457600-4915199` header. Node′s TLS module encrypts it into a record. The kernel′s TCP adds source port, destination port 443, sequence and acknowledgement numbers and the receive window, and IP adds source and destination addresses. The source address is `10.3.19.42` because Plexo passed it as `localAddress`. The Wi-Fi driver adds the 802.11 header and frame check sequence. The frame goes out of `wlp0s20f3` because Plexo bound the socket to that device. So Plexo authors layer 7, controls the lifetime and number of layer-4 connections, and steers layer 3, without building any header below HTTP itself.

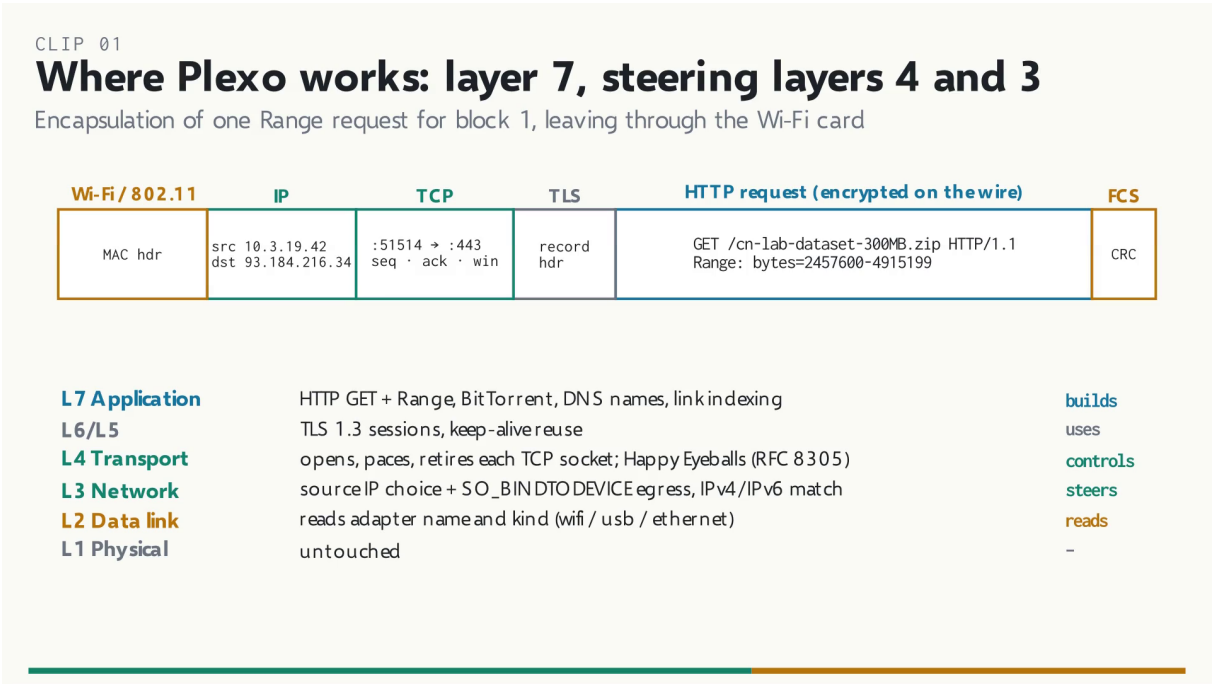


Figure 20. One Range request, encapsulated. Plexo builds the HTTP part, controls the TCP socket, chooses the IP source and the egress device, and only reads layer 2. *Source: clip 01-layer7-encapsulation.mp4 (authors, Manim).*

9.2 Socket programming, binding and DNS

Being multi-homed is not enough: the routing table still sends every packet out of the default route. Windows and macOS pick the egress interface from the source address, so binding `localAddress` is enough there. Linux routes by destination only, so a packet with the phone's source address would leave through Wi-Fi and be dropped by the Wi-Fi provider's ingress filter. `deviceBinding.ts` fixes this with a raw libc call through koffi: `socket(AF_INET, SOCK_STREAM | SOCK_CLOEXEC)`, then `setsockopt(fd, SOL_SOCKET, SO_BINDTODEVICE, "usb0")` (constants 1 and 25), then hands the file descriptor to a Node socket that connects with the same `localAddress`. Both bindings are needed, because one adapter can carry several IPv6 addresses. Name resolution uses `lookup(host, {all: true})` raced against a deadline. IPv4 and IPv6 attempts start 250 ms apart (Happy Eyeballs, RFC 8305 [3]), so a blackholed IPv6 path costs 250 ms rather than a 10 s timeout, and the winning route is remembered per network.

9.3 TCP and HTTP Range are different layers

HTTP Range and TCP are not interchangeable. Range is an application-layer header that names a byte interval; TCP [20] is the transport that delivers the request and the reply reliably and in order. Before any block is planned, a separate probe sends `GET` with `Range: bytes=0-1023`. Only a real `206 Partial Content` proves the server splits files, and `Content-Range: bytes 0-1023/314572800` gives the total size. `HEAD` is not trusted, because some servers advertise `Accept-Ranges` on HEAD and ignore Range on GET. Each download stream then pays one TCP three-way handshake and one TLS 1.3 handshake [21]. After that it sends Range request after Range request on the same kept-alive connection, which is connection pooling. DNS, the handshakes and TCP slow start are paid once per stream, not once per 2.3 MiB block.

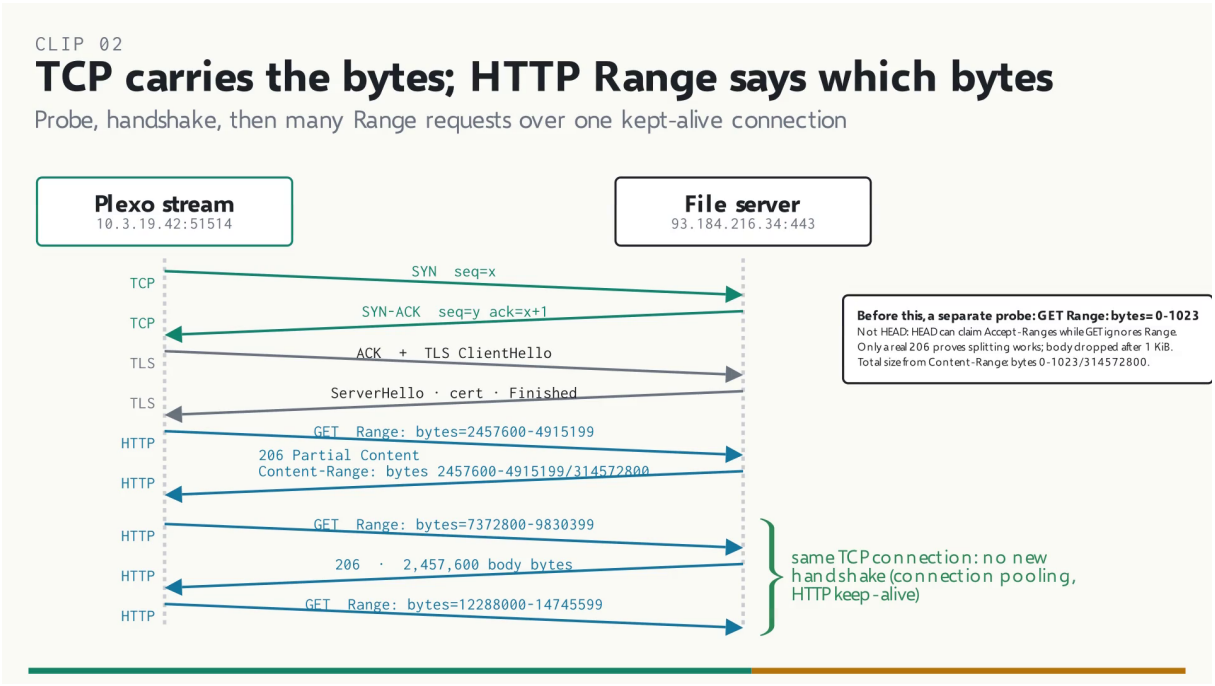


Figure 21. One pinned stream: TCP and TLS handshakes once, then many Range requests and 206 replies on the same connection. *Source: clip 02-tcp-vs-http-range.mp4 (authors, Manim).*

9.4 How a file is divided into chunks

`planDownload()` in `plan.ts` aims for 64 blocks per network (32 streams × 2 blocks each). For our 300 MiB file on two networks the target is 2 × 32 × 2 = 128 blocks. The block size is ceil(314,572,800 ÷ 128) = 2,457,600 bytes (2.34 MiB), clamped between 1 MiB and 8 MiB, giving 128 blocks. Block k is the request `Range: bytes=k×2457600-(k+1)×2457600−1`, about 1,684 TCP segments at a 1,460-byte MSS. A 1 GB file on two networks hits the 8 MiB cap (128 blocks). A 100 MB file hits the 1 MiB floor (100 blocks), and a server without range support gets one open-ended block on one stream. The cap bounds the cost of re-fetching a slow last block; the floor keeps per-request overhead small; two blocks per stream mean no stream waits idle. A block is never a separate file, only an interval of the one staging file (Section 9.6).

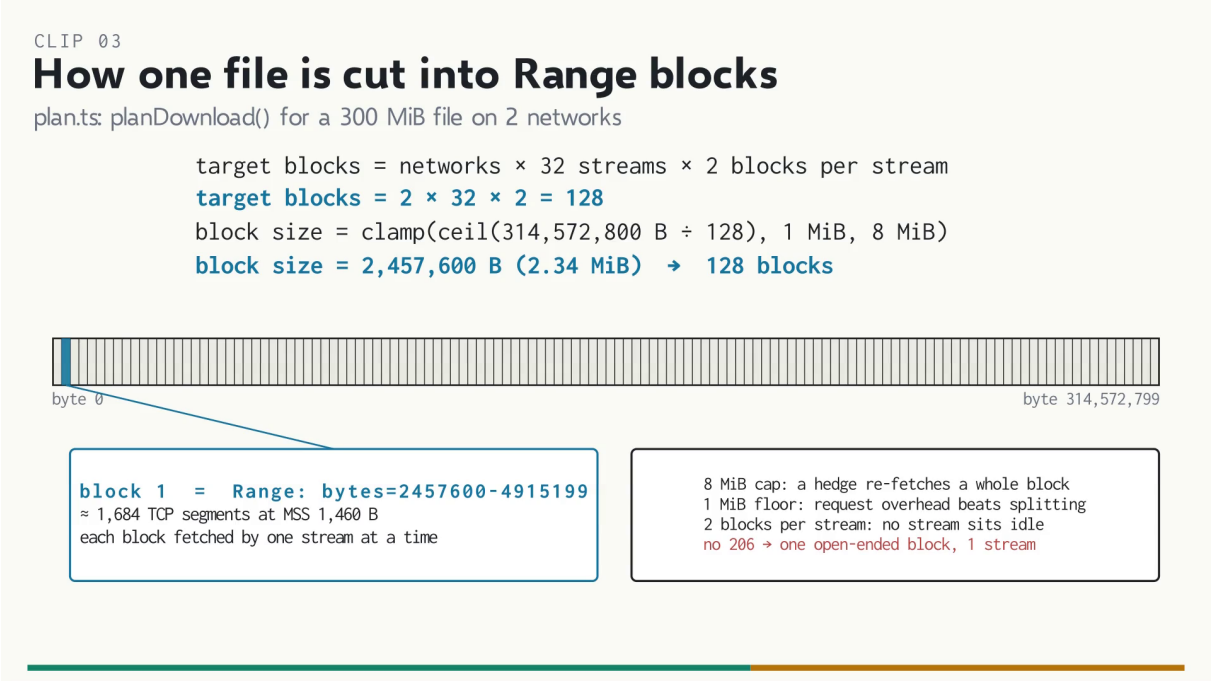


Figure 22. The 300 MiB file cut into 128 Range blocks of 2,457,600 bytes, with block 1's byte range and the planner's limits. *Source: clip 03-how-chunks-are-divided.mp4 (authors, Manim).*

9.5 Load balancing, the tail and integrity

Plexo never assigns a fixed share of the file to a network. Each stream that finishes a block asks the scheduler for the next pending one, so the faster path simply asks more often. That is how our bench split 62.5 : 37.5 against a 64 : 36 capacity ratio without knowing either limit. The concurrency controller starts 8 streams per network and doubles while every stream has answered, up to 32. If a server refuses some streams with 403, 429 or 503 while serving others, it cuts the ceiling to the number served, then recovers by one stream per minute. At the end, an idle stream may hedge the slowest block from its frontier. It does so only if nothing is pending, the attempt is older than 2 s, there are fewer than 2 hedges, the disk keeps up, and the holder would take at least twice as long [8]. Before any byte is written, the response must be 206 (or 200 at byte 0), Content-Range must start where asked, extra bytes are cut, and ETag, Last-Modified and size must match. Progress is the highest secured offset, never a sum.

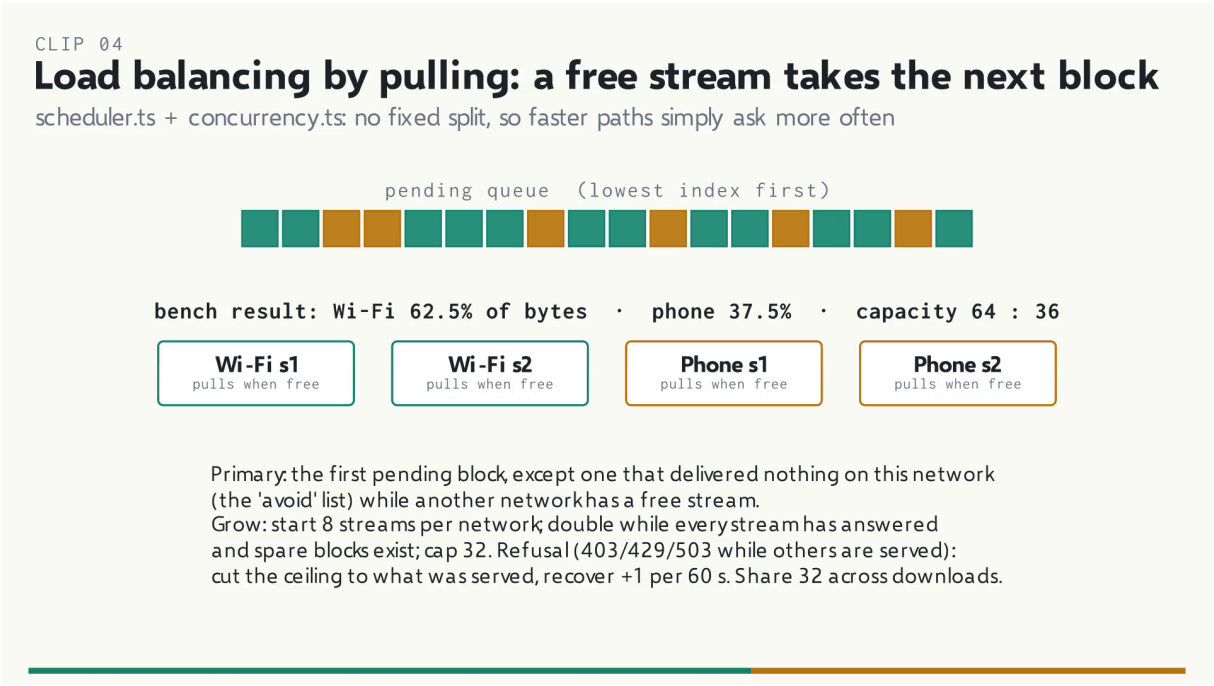


Figure 23. Pull-based scheduling: whichever stream is free takes the next block, so faster paths take more blocks. *Source: clip 04-load-balancing-scheduler.mp4 (authors, Manim).*

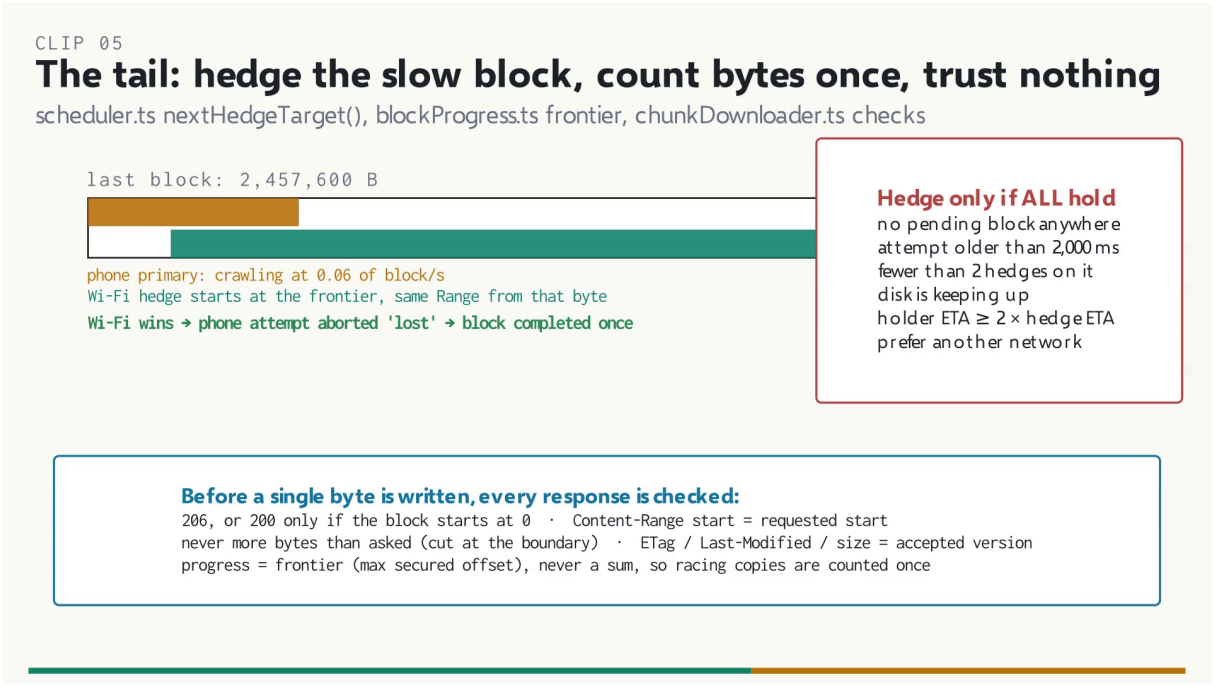


Figure 24. The tail: a Wi-Fi hedge races the slow phone block from its frontier and wins, and every response is checked before it is written. *Source: clip 05-tail-hedging-integrity.mp4 (authors, Manim).*

9.6 Why chunk downloads can need twice the disk, and why Plexo does not

A simple multi-part downloader saves each chunk as its own file (part001 to part128) and joins them when all are done. During the join the parts and the growing final file exist together. For our 300 MiB file, peak use is 300 MiB of parts plus 300 MiB of final file: 600 MiB, twice the file. Plexo avoids this with one staging file, `name.zip.plexo`, beside the final name. Every attempt opens its own write stream at an explicit offset, `createWriteStream(path, {flags: 'r+', start: offset})`, so blocks

land in place in any order. Writes are buffered up to 512 KiB per stream and flushed as one `writev`. When every block is done, Plexo calls `fsync` and renames the staging file to the final name in the same folder. The rename is a metadata change: no byte is copied, so the peak is about one file plus a manifest of a few kilobytes.

Two details keep this true. First, `ensureDiskSpace()` checks `statfs` free blocks against the file size, not twice the size. Second, auto-organize chooses the category folder before the staging file is created, so the final step is still a same-folder rename and never a cross-folder copy. On Windows the staging file is marked sparse with `FSCTL_SET_SPARSE`, because NTFS would otherwise write zeros into every gap before a scattered write, writing the file twice. The pre-download simulation shows peak storage as 1.08 × the file size (324 MiB for our file). That factor is a fixed safety margin in `simulationEngine.ts`, not a measured need: the real peak is about 1.0 ×.

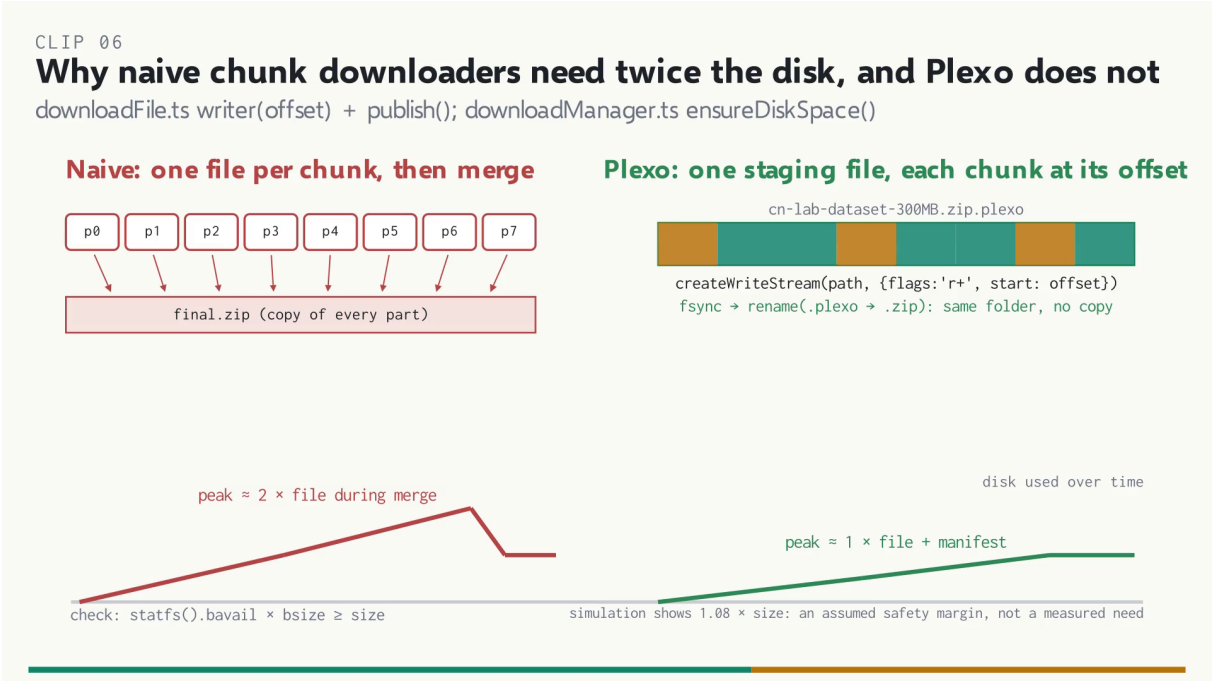


Figure 25. Separate chunk files peak at about twice the file during the merge; Plexo's single staging file plus rename peaks at about one file. *Source: clip 06-why-twice-the-disk.mp4 (authors, Manim).*

9.7 Flow control, limits and fault tolerance

Speed limits are token buckets: `take(n)` refills `min(r, tokens + elapsed × r)`, subtracts n, and returns a wait of `−tokens ÷ r` seconds when the bucket is in debt. The bucket holds at most one second of tokens, so idle time never banks a burst. While Plexo waits it calls `res.pause()` and stops reading. The kernel's receive buffer fills, the advertised TCP window falls to zero, and the sender stops by itself. Nothing is dropped or retransmitted. This is receiver-side flow control [13], [20], separate from TCP's congestion control. For faults, a NetworkMonitor polls the interface list every second. When the phone disappears its streams retire, its in-flight blocks return to the queue with their secured bytes kept, and a Wi-Fi stream resumes each one with a Range from its frontier. Retries back off from 1 s, doubling with ±20% jitter up to 15 s. Retry-After is honoured up to 120 s, and a network silent for 5 s while others serve is marked unreachable and probed by one stream.

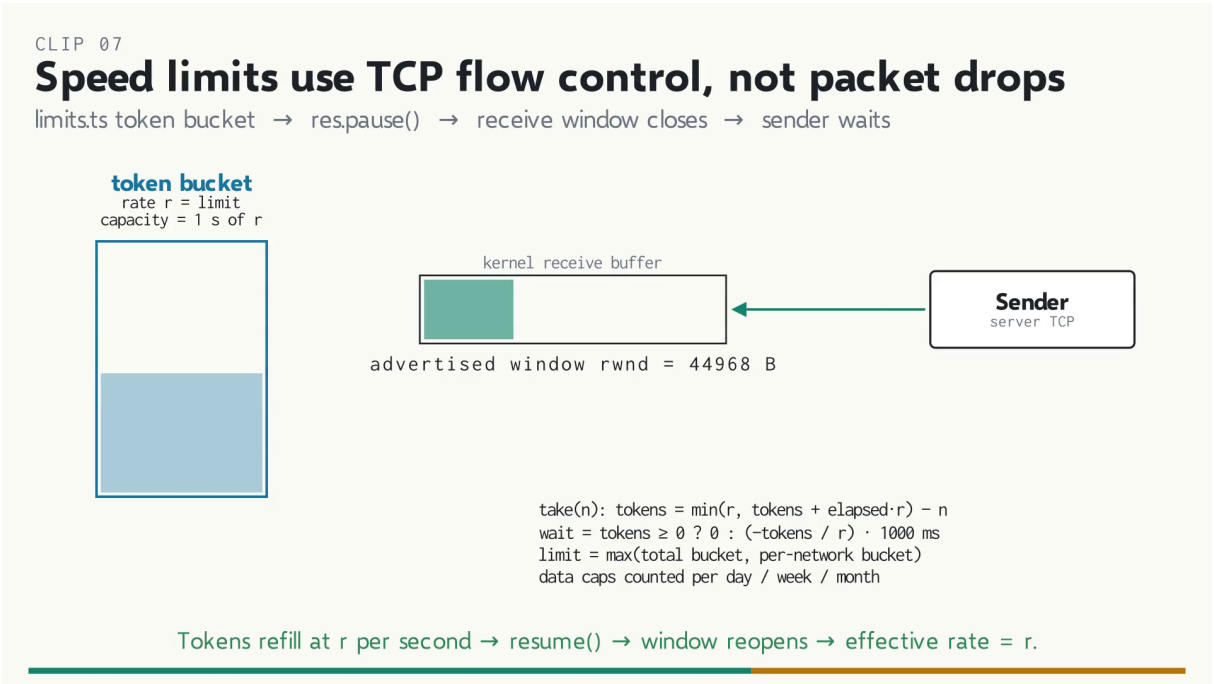


Figure 26. A speed limit pauses reading; the TCP receive window closes and the sender waits, with no packet loss. *Source: clip 07-tcp-flow-control-limits.mp4 (authors, Manim).*

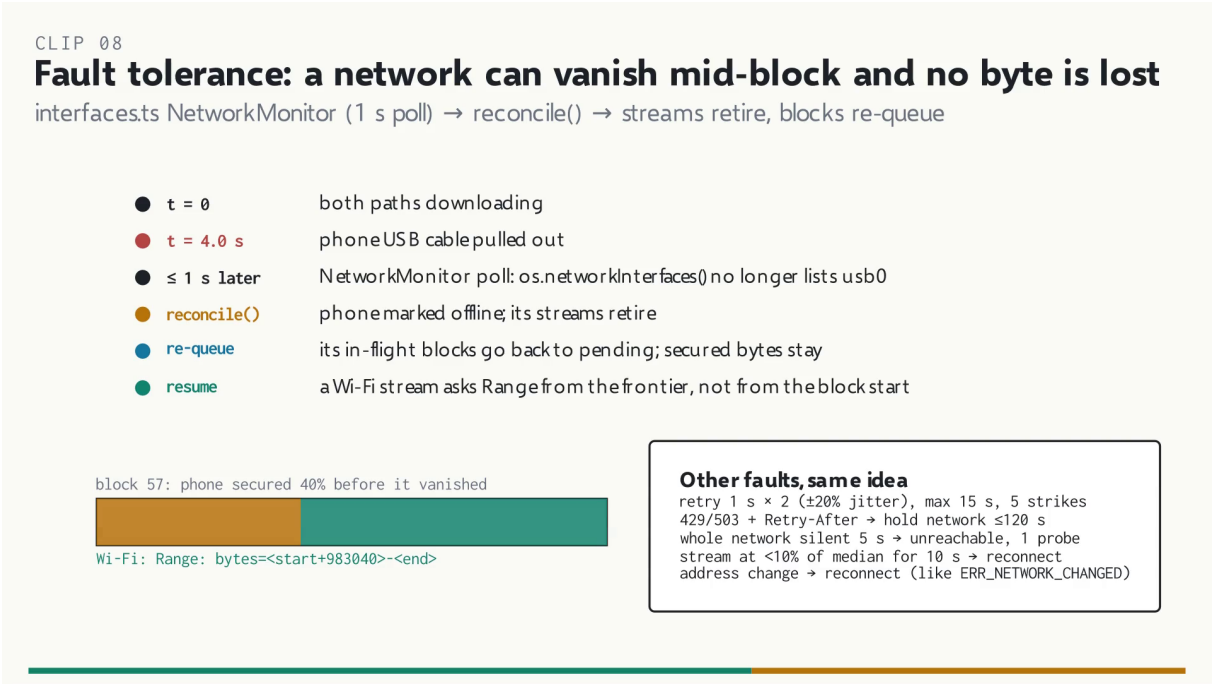


Figure 27. When a network vanishes mid-block, its blocks re-queue and another network resumes them from the frontier. *Source: clip 08-fault-tolerance.mp4 (authors, Manim).*

9.8 Peer-to-peer and security boundaries

For torrents, a two-hunk patch to webtorrent routes every outgoing peer connection through one hook. The hook sends each new peer to the network with the fewest peers (up to 30 per network) and pins its TCP socket there. Pieces count only after their SHA-1 hash checks. uTP is off because UDP sockets cannot be pinned to one network. The security model has four boundaries. The React window has no network or file access and calls 38 typed IPC channels that the main process

validates. The browser-extension bridge listens only on 127.0.0.1:19288–19290. It rejects a wrong Host header (DNS rebinding), a non-extension Origin (CORS) and non-JSON bodies, which force a preflight a web page cannot pass. TLS verifies every certificate against the requested host [21]. The SSRF guard keeps pasted index URLs away from private addresses [17], [18]. Plexo uses no WebSockets and has no proxy support.

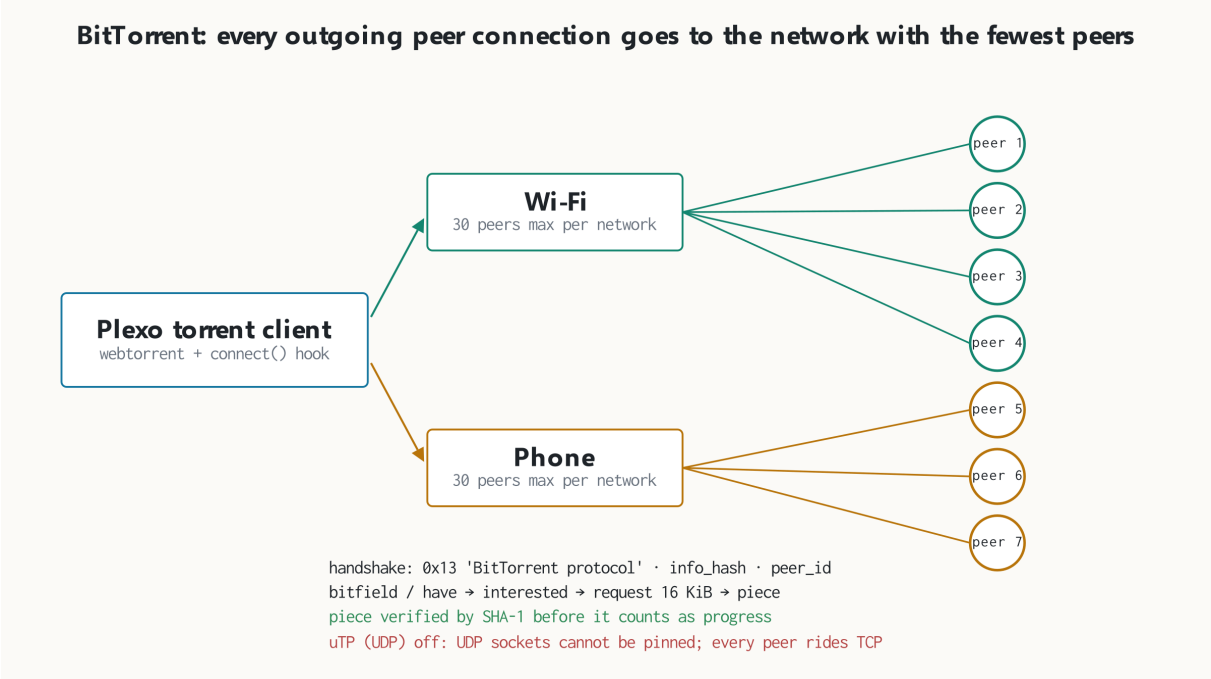


Figure 28. Every outgoing BitTorrent connection goes to the network with the fewest peers; all peers use TCP. *Source: authors, from download/torrent/ and patches/webtorrent+3.0.21.patch.*

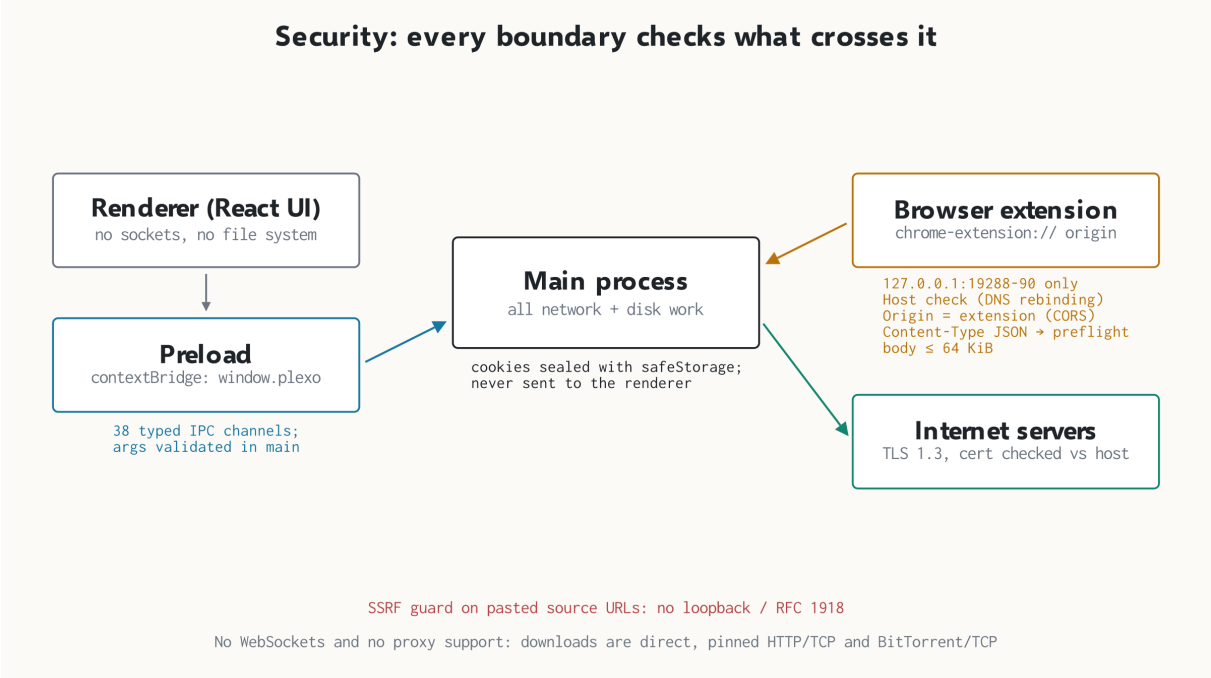


Figure 29. Each trust boundary checks what crosses it: IPC, the localhost bridge, TLS and the SSRF guard. *Source: authors, from src/preload, src/main/browserBridge.ts, discovery/sourceIndex.ts.*

9.9 Link indexing, step by step

When the user pastes https://ftp.gnu.org/gnu/hello/, the SSRF guard runs first. The adapter is chosen from the URL, and `robustFetch()` tries Electron's Chromium network stack, then curl, then Node's fetch, 8 s each. The universal adapter crawls breadth-first, at most 8 pages or 350 items. On each page it tries JSON-LD, Next.js data, card markup, WordPress entries, table listings, link lists and direct file links in that order; RSS and sitemaps are parsed too. The GNU page is an Apache table listing, which gave 45 items saved to `content-cache.json`. When the user picks one, its direct link goes through the normal probe, so it enters the same checked pipeline as any pasted download link.

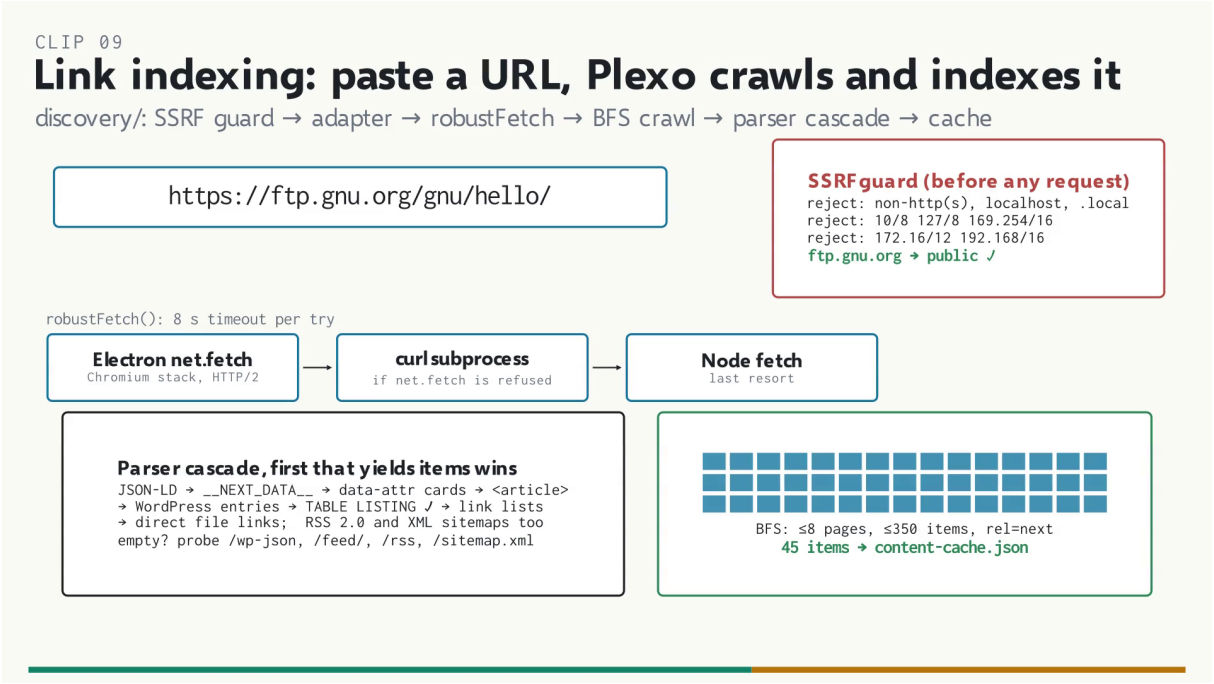


Figure 30. Link indexing: SSRF guard, fetch fallback chain, parser cascade and the 45 cached items. *Source: clip 09-link-indexing.mp4 (authors, Manim).*

10 Conclusion

We extended Plexo, an application-layer multi-path download manager, with seven enhancements: auto-organize, a Thompson Sampling tail strategist, a sharper translucent interface, a per-path speed test, link indexing with an SSRF guard, a pre-download simulation, and a designed checksum repair loop. We built each one on Plexo's existing guarantees (pinned sockets, range checks, frontier accounting) rather than around them, and verified them with 31 automated checks, a reproducible two-path bench and a real Wi-Fi measurement. On the bench one file moved at 73.1 MB/s over two paths, 1.6 times the faster path alone, and the bytes split in proportion to capacity without the app knowing either limit. For people on a slow or capped connection with a phone in their pocket, that means faster and more predictable downloads from a pasted link. The work that should come next follows from Section 8: implement the checksum repair, feed measured speeds into the simulation, make the crawler respect robots.txt, and harden the SSRF guard at the DNS level.

References

[1] A. Kapil and contributors, “Plexo: multi-network download manager,” source code, v1.0.0-rc.15, GitHub, github.com/anmolkapil/plexo, 2026.

[2] R. Fielding, M. Nottingham and J. Reschke, “HTTP Semantics,” RFC 9110, IETF, Jun. 2022 (Section 14, Range Requests).

[3] D. Schinazi and T. Pauly, “Happy Eyeballs Version 2: Better Connectivity Using Concurrency,” RFC 8305, IETF, Dec. 2017.

[4] A. Ford, C. Raiciu, M. Handley, O. Bonaventure and C. Paasch, “TCP Extensions for Multipath Operation with Multiple Addresses,” RFC 8684, IETF, Mar. 2020.

[5] C. Raiciu, C. Paasch, S. Barré, A. Ford, M. Honda, F. Duchene, O. Bonaventure and M. Handley, “How Hard Can It Be? Designing and Implementing a Deployable Multipath TCP,” in Proc. 9th USENIX NSDI, San Jose, CA, 2012, pp. 399–412.

[6] D. Wischik, C. Raiciu, A. Greenhalgh and M. Handley, “Design, Implementation and Evaluation of Congestion Control for Multipath TCP,” in Proc. 8th USENIX NSDI, Boston, MA, 2011.

[7] D. Kaspar, K. Evensen, P. Engelstad and A. F. Hansen, “Using HTTP Pipelining to Improve Progressive Download over Multiple Heterogeneous Interfaces,” in Proc. IEEE ICC, Cape Town, 2010, doi: 10.1109/ICC.2010.5502574.

[8] J. Dean and L. A. Barroso, “The Tail at Scale,” Communications of the ACM, vol. 56, no. 2, pp. 74–80, Feb. 2013.

[9] S. Agrawal and N. Goyal, “Analysis of Thompson Sampling for the Multi-armed Bandit Problem,” in Proc. 25th Conf. on Learning Theory (COLT), JMLR W&CP vol. 23, 2012, pp. 39.1–39.26.

[10] O. Chapelle and L. Li, “An Empirical Evaluation of Thompson Sampling,” in Advances in Neural Information Processing Systems 24 (NIPS), 2011, pp. 2249–2257.

[11] B. P. Welford, “Note on a Method for Calculating Corrected Sums of Squares and Products,” Technometrics, vol. 4, no. 3, pp. 419–420, 1962.

[12] N. Feamster and J. Livingood, “Measuring Internet Speed: Current Challenges and Future Recommendations,” Communications of the ACM, vol. 63, no. 12, pp. 72–80, Dec. 2020.

[13] V. Jacobson, “Congestion Avoidance and Control,” in Proc. ACM SIGCOMM ’88, Stanford, CA, 1988, pp. 314–329.

[14] B. Cohen, “Incentives Build Robustness in BitTorrent,” in Proc. 1st Workshop on Economics of Peer-to-Peer Systems, Berkeley, CA, 2003.

[15] A. Heydon and M. Najork, “Mercator: A Scalable, Extensible Web Crawler,” World Wide Web, vol. 2, no. 4, pp. 219–229, 1999.

[16] M. Koster, G. Illyes, H. Zeller and L. Sassman, “Robots Exclusion Protocol,” RFC 9309, IETF, Sep. 2022.

[17] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot and E. Lear, “Address Allocation for Private Internets,” RFC 1918, IETF, Feb. 1996.

[18] OWASP Foundation, “A10:2021 – Server-Side Request Forgery (SSRF),” OWASP Top 10, 2021.

[19] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015.

[20] W. Eddy, Ed., “Transmission Control Protocol (TCP),” RFC 9293, IETF, Aug. 2022.

[21] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, IETF, Aug. 2018.

Appendix A Where this report meets the rubric

Table 8. CIE 3 rubric criteria and the evidence for each

Rubric criterion	Marks	Evidence in this report
Understanding of problem statement	2	Sections 1, 2, 5
Execution: optimized code, validations, test cases	2	Sections 6, 7.5; Table 5; Appendix B
Results and documentation	2	Section 7 (six proofs), Tables 4–5, comments cited by file
Conceptual understanding (viva)	2	Sections 2, 3, 5, 9; Appendix C
Communication of networking concepts (viva)	2	Section 9 (Figures 20–30), ten clips, deck
Report submission (group project)	5	Whole report, references, known issues
Project demonstration	5	Section 7; live demo on the bench in Section 3
Presentation and viva	5	Deck (15 slides) + 10 technical clips; Appendix C

Appendix B Test log excerpt

```
$ npm test
✓ Feature 1 (TS): extensionOf handles edge cases (1.36ms)
✓ Feature 1 (TS): resolveOrganizeDir safeguards against path traversal (0.34ms)
✓ Feature 1 (Python): scripts/organize.py dry-run works as expected (84.55ms)
✓ Feature 2: Bandit arm selection explores untried arms first (0.72ms)
✓ Feature 2: Rewards move beliefs toward the higher performing arm (0.38ms)
✓ Feature 2: Learning store handles LRU eviction and corruption recovery (19.54ms)
✓ Feature 3: Window material transparency and rebuild triggers (0.54ms)
✓ Feature 4: SpeedTestRun instantiation, cancellation, and empty network handling (1.24ms)
ℹ tests 13  ℹ pass 13  ℹ fail 0
Testing SourceIndexManager & SSRF Protection...
  Actual error caught: Private, loopback, or non-HTTP URLs are prohibited
  ✓ SSRF rejection passed for local/private IPs
ALL SOURCE INDEX AND DISCOVERY TESTS PASSED!
  [PASS] Multi-interface bonding speedup and peak storage calculation verified
ALL PRE-DOWNLOAD SIMULATION TESTS PASSED!
```

Appendix C Short answers for the viva

Which layer is Plexo on?

Layer 7. It controls layer-4 TCP sockets and steers layer-3 egress by source IP and `SO_BINDTODEVICE`; it only reads layer-2 adapter names.

Why not Multipath TCP?

MPTCP needs kernel support on both ends [4], [5]. Plexo needs only HTTP range support on the server, which most file hosts already have [2].

Why probe with bytes 0–1023 instead of HEAD?

HEAD can claim range support that GET ignores. Only a real 206 on a real GET proves splitting works.

How does a speed limit slow the sender without dropping packets?

Plexo stops reading; the receive buffer fills, the TCP window closes and the sender waits. That is flow control [13].

What does Thompson Sampling add?

It picks the tail arm in proportion to the chance it is best, so it keeps exploring uncertain arms and settles on good ones [9], [10].

What stops link indexing from attacking the local network?

The SSRF guard refuses loopback, link-local and RFC 1918 targets before any request [17], [18]. Section 8 lists its gaps.

When does bonding give no gain?

When both adapters share one upstream link, or the server refuses ranges, or a torrent has a single peer.